



SEMESTER 2 REPORT

29/4/2026

Deepfake Voice Detection Using Deep Learning with Real-Time WhatsApp Integration

(20101947) Christopher Kelleher

Bachelor of Science (Honours) Computer
Science (Computer Forensics & Security)

Contents

Generative AI Statement	6
Abstract	7
1. Introduction	8
2. Literature Review	10
2.1 Synthetic Voice and Deepfake Detection.....	10
2.2 AI and Sequential Data Processing	10
2.3 Real-World Implications of Synthetic Voice Attacks	11
3. Methodology	12
3.1 Agile Workflow	12
3.2 Project Management Tools	12
3.3 Research and Tools	13
4. System Architecture	14
4.1 Evolution of the System Architecture	14
4.2 Overall System Design	16
4.3 WhatsApp Chatbot Interface.....	16
4.4 Web Interface	18
4.5 Backend API (FastAPI on AWS EC2)	19
4.6 Audio Processing Pipeline.....	19
4.7 Machine Learning Model Deployment.....	20
4.8 Training Environment	20
4.9 Architectural Considerations and Design Decisions.....	21
5. System Implementation.....	22
5.1 Overview of Implementation Approach	22
5.2 Web Interface Implementation	22
5.2.1 File Handling and User Input	22
5.2.2 Audio Preview and Feedback	23
5.2.3 Audio Recording Pipeline.....	23
5.2.4 API Communication	24

5.3 Backend API Implementation (FastAPI)	25
5.3.1 Model Loading and Device Management.....	25
5.3.2 Windowing Segmentation	26
5.3.3 API Endpoint Design.....	26
5.3.4 File Handling and Audio Conversion	27
5.3.5 Audio Augmentation	28
5.3.6 Feature Extraction.....	28
5.3.7 Model Inference and Decision Logic	29
5.4 WhatsApp Bot Implementation	30
5.4.1 Authentication and Session Handling.....	30
5.4.2 Event-Driven Message Handling.....	31
5.4.3 API Communication	31
5.4.4 Response Formatting	32
5.5 Cloud Implementation	33
5.6 Ngrok for Development	35
6. Model Training and Development.....	36
6.1 Data Exploration for ASVspoof.....	36
6.1.1 Dataset Structure and Label Analysis	36
6.1.2 Loading and Inspecting Audio Samples	37
6.1.3 Spectral Analysis using Log-Mel Spectrograms	39
6.1.4 Summary of Dataset Exploration	39
6.2 Wav2Vec2 Training Pipeline.....	40
6.2.1 Environment Setup, Imports and GPU Verification	40
6.2.2 Dataset Loading and Multi-Source Integration.....	41
6.2.3 Audio Loading, Normalisation and Feature Preparation	42
6.2.4 Model Loading and Architecture Configuration	46
6.2.5 Data Loading and Class Imbalance Handling	47
6.2.6 Optimisation Strategy and Evaluation Metrics	49
6.2.7 Training Loop and Model Optimisation.....	51

6.3 LSTM Model Training and Development.....	53
6.3.1 Mel-Spectrogram Feature Representation	53
6.3.2 Dataset Adaptation for Spectrogram	54
6.3.3 LSTM Model Architecture Design.....	55
6.3.4 Training Configuration and Data Loading Differences.....	56
6.3.5 Training Loop and Evaluation Differences	56
6.4 PANNs CNN14 Model Selection and Setup	57
6.4.1 Input Representation Difference	57
6.4.2 Dataset Design Differences	58
6.4.3 Model Forward Pass and Output Handling	58
7. Evaluation.....	59
7.1 Model Evaluation	59
7.1.1 Evaluation Metrics Overview.....	59
7.1.2 Wav2Vec2 Model Performance Analysis	60
7.1.3 LSTM Model Performance Analysis.....	62
7.1.4 CNN14 (PANNs) Model Performance Analysis.....	64
7.1.5 Overall Findings	67
7.2 System Evaluation.....	68
7.2.1 WhatsApp Integration Performance.....	68
7.2.2 Web Interface Performance (High-Quality Audio Testing)	71
7.2.3 Impact of Audio Quality and Compression.....	71
7.2.4 Effectiveness of Windowing Strategy	72
7.2.5 System-Level Summary.....	72
8. Conclusion.....	73
9. Future Work.....	74
Glossary.....	75
Bibliography	80

Figure 1 FYP Gannt Chart	12
Figure 2 GitHub Commits	13
Figure 3 First Architecture design using Machine Learning (ML) Model.....	14
Figure 4 Final System Architecture	15
Figure 5 Sample of WhatsApp Interface	17
Figure 6 Web Interface	18
Figure 7 File Handling for user interface	22
Figure 8 Preview function	23
Figure 9 In Browser Recording	23
Figure 10 uploadFile	24
Figure 11 Model Loading	25
Figure 12 Windowing Function	26
Figure 13 FFmpeg Conversion	27
Figure 14 Audio Augmentations	28
Figure 15 Feature Extractor	28
Figure 16 Model Inference.....	29
Figure 17 Setup and QR Code Generation	30
Figure 18 Event-Driven Handling.....	31
Figure 19 Bot to API Communication	31
Figure 20 User Response Formatting	32
Figure 21 EC2 Configuration.....	34
Figure 22 WhatsApp Bot running on EC2	35
Figure 23 Model Loaded on EC2	35
Figure 24 ngrok tunnel output.....	35
Figure 25 ngrok http requests	35
Figure 26 Dataset and Labels	36
Figure 27 Visualizing Audio Samples	37
Figure 28 Real vs Fake waveform	38
Figure 29 Log-Mel Real vs Fake Visualized	39
Figure 30 GPU config and Imports	40
Figure 31 Loading ASVspoof data and assigning labels to integers	41
Figure 32 Merging WhatsApp Voice Notes	42
Figure 33 Dataset Loading Output	42
Figure 34 Audio Loading.....	43
Figure 35 Opus Compression	44
Figure 36 Augmentation Functions	45
Figure 37 Fixed Length Function	45
Figure 38 Feature Extraction and Normalisation	46

Figure 39 Model Loading and Layer Freezing.....	47
Figure 40 Weighted Sampling	49
Figure 41 Class Weight Values.....	49
Figure 42 Optimiser	50
Figure 43 Full Training Loop	52
Figure 44 Mel-Spectrogram Transformation	53
Figure 45 Mel-Spectrogram on the Dataset	54
Figure 46 LSTM Model Code	55
Figure 47 Weighted Sampler for LSTM	56
Figure 48 Updated Evaluation for LSTM training	57
Figure 49 CNN Audio Augment	58
Figure 50 Cross entropy for loss CNN	59
Figure 51 Wav2Vec Loss Curve	60
Figure 52 Wav2Vec Accuracy Curve	61
Figure 53 Wav2Vec ROC-AUC Curve	61
Figure 54 LSTM Loss Curve	63
Figure 55 LSTM Accuracy Curve.....	63
Figure 56 ROC-AUC Curve LSTM.....	64
Figure 57 CNN Loss Curve	65
Figure 58 CNN Accuracy Curve	65
Figure 59 ROC-AUC CNN Curve	66
Figure 60 Comparative Loss Curve	67
Figure 61 Comparative Accuracy Curve	68
Figure 62 Comparative ROC-AUC Curve	68
Figure 63 Uncompressed Audio Visualisation.....	70
Figure 64 WhatsApp audio Visualisation	70

Generative AI Statement

Generative artificial intelligence (GenAI) was used in this work. I, the author, have reviewed and edited the content as needed and take full responsibility for the content, claims, and references. An overview of the use of GenAI is provided below.

I used OpenAI's ChatGPT for work completed in this report: <https://chatgpt.com/>

- To assist with grammar and tone of some segments for better sentence structure. The tool was used to provide better wording, but all content was original and written by me.
- To brainstorm the general layout and structure of the report i.e. where in the report I should place headings and what information I should include
- To assist in gathering possible confusing keywords for unfamiliar readers in a list. I used that list to create the glossary of terms found in the report.
- To find areas of the report that were lacking in referencing. The references were not provided by the AI, only to identify sentences that made a claim or made mention of a technology but was missing a reference

I used Anthropic's Claude for work completed in my code: <https://claude.ai/new>

- To assist with reading error messages and debugging but not code generation fixes. Sample Prompt: "Could you look at this error message and this piece of code (insert code) and let me know in plain English what might be happening?"
- To assist with adding features and functions previously researched or found in documentation, but not the generation of code for these functions. Sample prompt: "Could you explain in English how I could add cross entropy loss for classification to this function so that I can calculate the loss for the model during training?"
- To assist with improving code comments and suggesting areas where code comments could be useful. Actual original code comments were written by me. Sample prompt: "Could you look at this functions code comment and make it clearer to a reader? (insert code comment)". "Could you look at this section of code and let me know if it would benefit from code comments?"

Abstract

The rapid advancement of artificial intelligence (AI) has greatly improved the quality and the availability of synthetic voice generation. This has increased the risk of voice related social engineering attacks such as voice phishing or vishing. This project presents the design, implementation, and evaluation of an AI system for detecting synthetic or deepfake voice messages through an accessible messaging platform. The system integrates a WhatsApp chatbot interface with a FastAPI (FastAPI, 2023), backend and a deep learning model for audio classification. This enables users to submit voice notes for automated analysis.

The solution was developed using the WhatsApp-Web library (wwebjs.dev, n.d.), to interface with WhatsApp Web. This allows for seamless reception and transmission of voice messages, although a real-world solution would likely use the dedicated WhatsApp Application Programming Interface (API) for business use. Audio files are processed through a backend pipeline that performs format conversion and augmentation using FFmpeg (Ffmpeg.org, 2018) to ensure consistency. A fine-tuned Wav2Vec2 model was implemented for binary classification of real versus synthetic speech, trained on a combination of publicly available datasets, including the Automatic Speaker Verification Spoofing (ASVspoof) 2019 dataset (Yamagishi et al., 2021), and augmented real-world WhatsApp voice samples.

To improve model generalisation and robustness to real-world conditions, extensive data augmentation techniques were applied. This included compression artefacts, background noise injection, bandpass filtering, and pitch variation. The system was evaluated using standard performance metrics such as accuracy, loss, and Receiver Operating Characteristic – Area Under the Curve (ROC-AUC), alongside practical testing using real WhatsApp voice messages and testing using the web interface. Results demonstrated that the model could distinguish between genuine and synthetic speech with a high degree of accuracy under studio conditions, but highlights challenges in generalisation to unseen synthesis techniques as well as poor quality and compressed audio which is common in the real world.

This project demonstrates the feasibility of integrating deep learning audio analysis into widely used communication platforms to provide users with accessible security tools. The findings highlight both the potential and limitations of current deepfake detection approaches and emphasise the importance of continuous model adaptation in response to evolving synthetic voice technologies. Future work will focus on improving model robustness, expanding datasets, and deploying the system within a scalable cloud-based infrastructure to support real-time analysis.

There is also a glossary contained in this report which contains the meaning of acronyms and explanations of technical terms which can be referred to while reading this report.

1. Introduction

Modern deep learning models are now capable of generating highly realistic human speech using only short audio samples, making synthetic voices increasingly accessible and more difficult to distinguish from genuine recordings (Jayakannan, 2025). These developments have allowed for beneficial applications such as accessibility tools, virtual assistants, and automated customer service systems. However, they have also introduced serious security concerns. One of the most prominent threats arising from this technology is voice phishing, commonly referred to as vishing, where attackers exploit synthetic or cloned voices to deceive individuals into revealing sensitive information or transferring funds (Nccgroup.com, 2025; Huntress, 2026).

Recent reports have shown a sharp increase in the use of deepfake audio in social engineering attacks, with synthetic voice technology being used to impersonate trusted individuals such as family members, colleagues, or senior executives (Nccgroup.com, 2025). These attacks are particularly effective because they exploit human trust in familiar voices and often happen in real-time communication environments, leaving a small margin of time for the victim to think and reflect on a potentially fraudulent message. Messaging platforms such as WhatsApp, which support voice note functionality, present a significant attack surface due to their widespread use and informal communication style. As the quality of synthetic speech continues to improve, traditional indicators of fraudulent audio, such as unnatural tone or distortion, are becoming less reliable, making detection increasingly challenging for the average user.

Existing solutions for detecting synthetic audio are typically designed for institutional or enterprise use, requiring specialised knowledge, dedicated infrastructure, or access to the proper tools. While these systems may achieve high accuracy in controlled environments, they are not easily accessible to everyday users who are most vulnerable to vishing attacks. This highlights a clear gap between advanced deepfake detection research and practical user-friendly applications that can be deployed for the public.

This project aims to address this gap by designing and implementing an accessible system that integrates deepfake voice detection into a widely used messaging platform. The core objective is to develop a tool that allows users to submit suspicious voice messages through a familiar interface and receive an automated analysis indicating the likelihood that the audio is synthetically generated. Rather than providing a definitive classification,

the system returns a confidence score, allowing users to make informed decisions based on AI-assisted insights rather than relying solely on their own subjective judgement.

To achieve this, a complete end-to-end cloud system was developed, consisting of a WhatsApp chatbot interface, a FastAPI (FastAPI, 2023) backend for handling audio processing and model inference, and a deep learning model for synthetic speech detection, all contained in an AWS environment (Amazon Web Services, 2025). The system processes incoming voice messages by converting and normalising the audio, applying augmentation techniques, and passing the data through a fine-tuned Wav2Vec2 model trained on both publicly available datasets and real-world audio samples. This design prioritises usability, allowing users to interact with the system without requiring technical knowledge or additional software.

The project also places a strong emphasis on evaluating the effectiveness of deepfake detection in realistic conditions. While many existing studies focus on performance using clean, studio-quality datasets, this work explores the challenges associated with real-world audio, including compression artefacts, background noise, and variability in recording quality. By using augmented data and testing with real WhatsApp voice messages, the project aims to better understand the limitations of current detection approaches and the extent to which they generalise beyond controlled environments.

The key objectives of this project are as follows:

- To design and implement an end-to-end system for detecting synthetic voice messages using a messaging platform interface.
- To develop and fine-tune a deep learning model capable of distinguishing between real and synthetic speech.
- To apply audio preprocessing and augmentation techniques to improve model robustness.
- To evaluate system performance using both standard metrics and real-world testing scenarios.
- To analyse the limitations of current deepfake detection methods and identify areas for future improvement.

This report presents the full development and evaluation of the system, including its architecture, implementation, methodology, and experimental results. It also discusses the broader implications of synthetic voice detection in cybersecurity and highlights the challenges that must be addressed to create reliable and scalable solutions in this ever-evolving field.

2. Literature Review

2.1 Synthetic Voice and Deepfake Detection

The detection of synthetic voices needs the understanding of both the temporal and spectral characteristics of audio signals. Synthetic voices often lack the natural variability, and patterns that are found in human speech.

Research by (Solanki, 2024) emphasizes the effectiveness of Recurrent Neural Networks (RNNs) for sequential audio analysis. RNNs, including Long Short-Term Memory (LSTM) models, can model data over time. This makes them good candidates for analysing speech patterns. Their system demonstrated the feasibility of detecting synthetic speech from audio files. However, their solution didn't have integration with messaging platforms, limiting accessibility and usability for users and instead went for a web-based Flask approach.

Research by (Chengzhe Sun, 2023) discusses another approach to synthetic voice detection by focusing on artifacts introduced during the neural vocoding stage of audio generation. Neural vocoders are an important component of text to speech and voice conversion systems. They are responsible for transforming abstract audio features like spectrographs into audible and high-quality speech. As genuine human speech does not undergo this process, vocoder artifacts can be a distinguishing feature for synthetic audio.

(Chengzhe Sun, 2023) introduce a multi-task learning framework based on the RawNet2 architecture, which works directly with raw audio waveforms. The model is trained on binary classification of real versus synthetic speech, and to identify the specific neural vocoder used to generate a synthetic sample. This encourages the model to learn more discriminative features related to vocoder artifacts which ultimately results in improved detection rates. Experimental results demonstrated strong accuracy across multiple datasets, including ASVspoof 2019, highlighting the effectiveness of this approach.

However, the authors also discussed limitations related to generalisation. Detection performance decreased when evaluating audio generated by unseen vocoders. Models trained on known synthesis techniques may struggle to adapt when new techniques are developed.

2.2 AI and Sequential Data Processing

Sequential data, such as audio needs models capable of maintaining contextual information across time. RNNs and some other variants like LSTM are quite effective for

this purpose. According to (Stryker, n.d.), RNNs can predict future values based on historical inputs, making them effective for anomaly detection in speech patterns.

The ability to learn long-term dependencies allows these models to capture subtle inconsistencies in timing, intonation, and spectral continuity in spectrographs that are often found in synthetic speech. The paper further explores this concept by testing multiple deep learning models for voice deepfake detection, including Convolutional Neural Networks (CNNs), LSTM networks (RNN), and transformer models.

Their findings indicate that while CNNs perform well on spectrogram inputs, transformer models achieved the highest detection accuracy (94.8%), followed closely by LSTM. This suggests that temporal modelling certainly plays an important role in identifying synthetic speech. The use of feature extraction techniques such as Mel-Frequency Cepstral Coefficients (MFCCs), log-Mel spectrograms, and chroma features further enabled effective detection.

2.3 Real-World Implications of Synthetic Voice Attacks

The increasing realism of synthetic voices poses significant security challenges. (Jayakannan, 2025) discusses how AI-generated speech can compromise voice authentication systems. This enables attackers to bypass verification mechanisms with high success rates. These vulnerabilities pose serious risks to financial institutions and individuals.

(Bajwa, 2025) provides multiple real world case studies showing the impact of deepfake technology on the public. During the 2024 U.S. Presidential Election, AI-generated robocalls pretending to be Joe Biden were used to discourage voter participation in the elections. These attacks truly showed how synthetic audio can be used to influence public behaviour and disrupt even governmental processes. Similar incidents with political figures and corporate executives have resulted in reputational damage and financial fraud. There are many cases where employees were tricked into transferring large amounts of money after hearing very real sounding deepfake audio impersonations of company leadership.

While much of the existing research focuses on institutional detection systems, this project aims to address the gap at the individual user level. By integrating deepfake analysis into a platform like WhatsApp, the system aims to give users AI-assisted confidence scores and actionable insights.

3. Methodology

This project followed an iterative, Agile methodology combining experimental development with structured research and continuous evaluation. The work was divided into two main parts:

- **Theoretical Investigation**
Research was conducted into deepfake audio detection, speech processing techniques, and machine learning architectures. Academic papers, existing models, and technical documentation were analysed to inform design decisions.
- **Experimental Development**
A series of experiments were carried out using Python (Jupyter Notebooks) to develop and test deep learning models. This included dataset preparation, feature engineering, model training, and evaluation using metrics such as accuracy, loss, and ROC-AUC.

3.1 Agile Workflow

An Agile-style iterative workflow was used to manage development. Each cycle involved:

- Planning experiments and model changes
- Implementing updates in code
- Testing model performance
- Evaluating results and refining the approach

This ensured continuous improvement across all models and system components.

3.2 Project Management Tools

Gantt Chart: Used to plan project phases and track deadlines

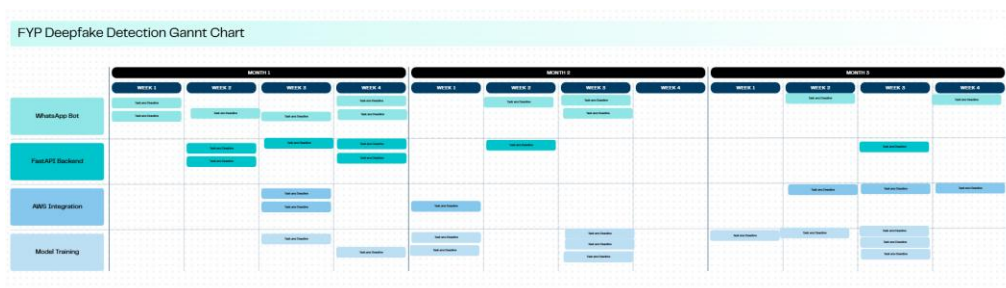


Figure 1 FYP Gantt Chart

GitHub: Used to software development tasks and monitor progress

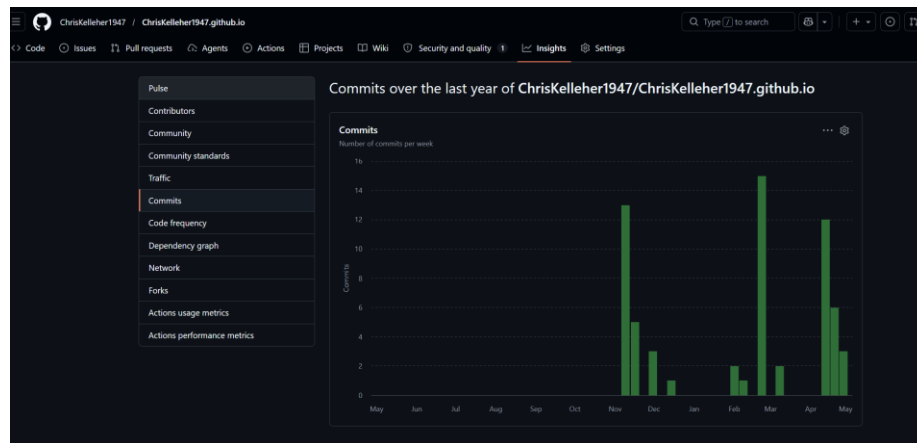


Figure 2 GitHub Commits

3.3 Research and Tools

Google Scholar: Academic research and paper sourcing

Zotero: Reference and literature management

4. System Architecture

4.1 Evolution of the System Architecture

The system architecture evolved significantly throughout the course of the project, much of which was due to both the progression of project objectives and practical challenges encountered during implementation. Rather than being designed as a complete solution from the start, the system was developed iteratively, with each stage building upon the previous one.

The initial version of the system was implemented as a proof-of-concept, focused on validating end-to-end communication between a user interface and an AI processing pipeline. At this stage, a WhatsApp chatbot was integrated with a FastAPI backend, and OpenAI's Whisper model was used for speech-to-text transcription (OpenAI, 2022). During this phase, the WhatsApp bot was implemented using the Baileys library (wwebjs.dev, n.d.). This version demonstrated that voice messages could be received, processed, and responded to automatically. However, it did not address the core objective of detecting synthetic speech and instead served as a foundation for further development.

The second iteration introduced a machine learning approach for deepfake voice detection. A fine-tuned Wav2Vec2 (ai.meta.com, n.d.), model was integrated into the system to classify audio as either genuine or synthetic. This marked a significant step forward, as the system was now capable of performing full end-to-end detection, including preprocessing, model inference, and confidence score generation.

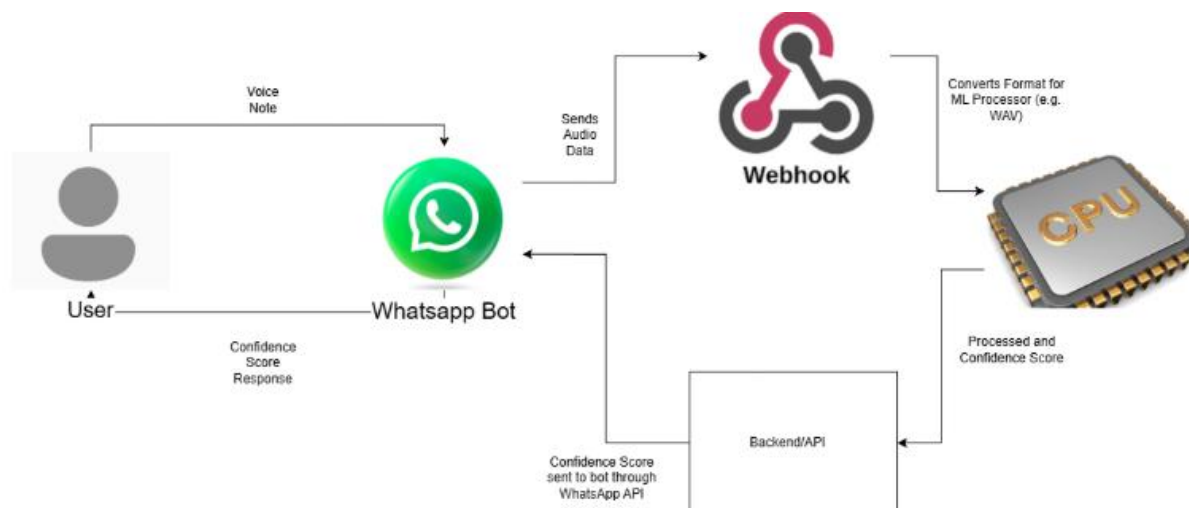


Figure 3 First Architecture design using Machine Learning (ML) Model

However, limitations emerged with this approach, particularly regarding the reliability of the Baileys library. Changes to WhatsApp Web protocols resulted in authentication and session stability issues, requiring frequent reconfiguration. These challenges highlighted the risks associated with relying on unofficial libraries for deployment.

The final iteration of the system introduced a cloud architecture to improve scalability and stability. The backend was deployed on an Amazon Amazon Elastic Compute Cloud (EC2) instance (Amazon Web Services, 2024), while the trained deep learning model was originally meant to be hosted using Amazon SageMaker as an inference endpoint (AWS 2025). However due to issues with SageMaker configurations, the model is also hosted on the EC2 instance. This works for demo purposes and is in line with the free tier, however a full production version of this project would most likely use SageMaker for both training and hosting. This is costly but a large scale deployment of the system would scale with this. The WhatsApp integration was adapted to use a more stable WhatsApp-Web, another service which offers connection to the WhatsApp Web official API. Additionally, a web interface was developed to support testing and demonstration purposes. This final architecture reflects a more realistic deployment model with clear separation between user interfaces, backend processing, and machine learning inference.

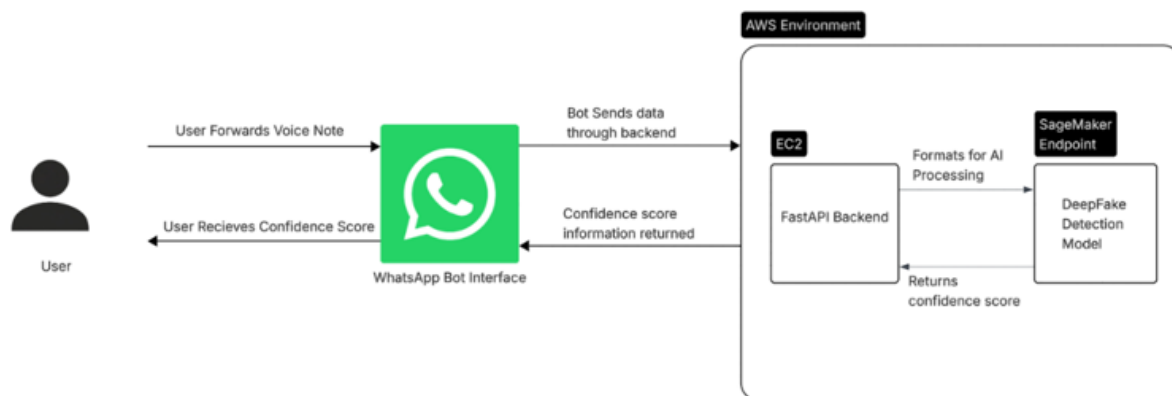


Figure 4 Final System Architecture

The following sections describe the final system architecture in detail, including its components, data flow and design considerations.

4.2 Overall System Design

The system was designed as a modular, end-to-end architecture. The main goal of the architecture was to ensure accessibility, scalability, and clear separation of responsibilities between components. It must also be compatible with messaging platforms.

The system consists of three core components:

1. A WhatsApp chatbot interface for user interaction / Web interface for testing and demonstration
2. A FastAPI backend hosted on a cloud server for processing
3. A deep learning model hosted separately for inference

The system uses a client–server architecture, where user interfaces act as clients that submit audio data to a main backend service. The backend is responsible for handling all processing tasks like audio conversion, preprocessing, and communication with the machine learning model. The model is hosted as a separate service to allow for modular deployment and scalability.

This separation between interface, backend, and model was a deliberate design decision. It ensures that each component can be developed and scaled independently. For example, the machine learning model can be updated or replaced without requiring changes to the WhatsApp bot or backend API, and additional interfaces could be integrated in the future with minimal modification to the core system.

4.3 WhatsApp Chatbot Interface

The primary user interface for the system is a WhatsApp chatbot implemented using a WhatsApp-Web library ([wwebjs.dev](https://github.com/whatsapp-web.js/whatsapp-web.js), n.d.). This allows the system to interact with WhatsApp without requiring access to the official business API, which involves setting up a business account and additional costs that are not suitable for this project.

The chatbot uses an event-driven model, continuously listening for incoming messages. When a user submits a voice note, the bot will do the following:

- Detects the presence of an audio message
- Downloads the audio file in its native format, typically .ogg for WhatsApp
- Forwards the file to the backend API via an Hypertext Transfer Protocol (HTTP) request

This design enables a seamless user experience, as users can simply forward a suspicious voice note to the chatbot without needing to install additional applications or manually upload files. The use of WhatsApp as the main interface significantly improves accessibility, as it takes advantage of an existing and widely adopted communication platform.

However, it is important to note that this implementation relies on the WhatsApp-Web interface rather than the official API. While this is suitable for development and demonstration purposes, a production system would require migration to the official WhatsApp Business API (Meta, n.d.; No-Code Start-Up, 2025), to ensure reliability and scalability.

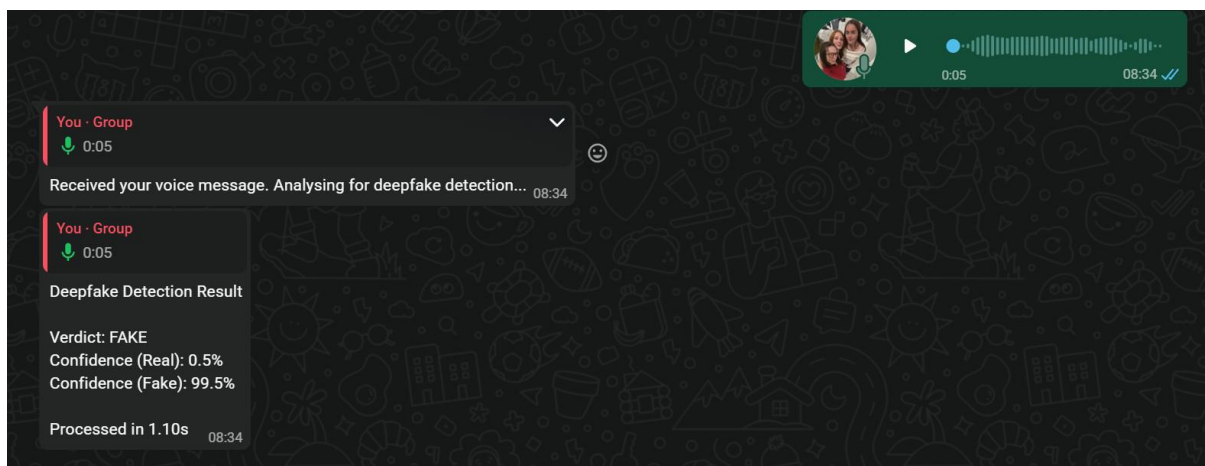


Figure 5 Sample of WhatsApp Interface

4.4 Web Interface

As well as the WhatsApp chatbot, a secondary web interface was developed to demonstrate and test the system. This interface is hosted using GitHub Pages and gives users the ability to upload audio files directly through a browser.

The web interface communicates with the backend via HTTP requests, like the WhatsApp bot. During development and demonstration, this interface was exposed using the tunnelling service ngrok, which allowed the locally hosted backend to be accessible over the internet.

While the web interface is not intended as the main deployment platform, it serves several important purposes:

- Provides an alternative method for testing the system without WhatsApp
- Allows easier debugging and inspection of responses
- Demonstrates the flexibility of the backend to support multiple client interfaces

This reinforces the idea of the modular nature of the architecture, where multiple front-end systems can interact with the same backend service.

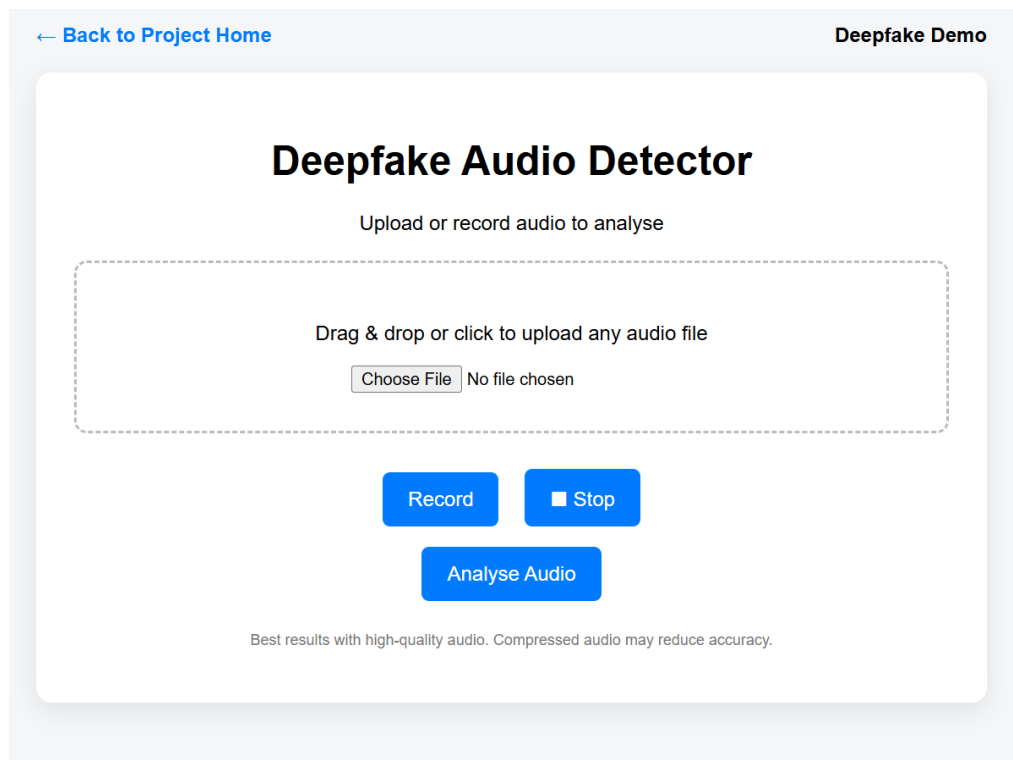


Figure 6 Web Interface

4.5 Backend API (FastAPI on AWS EC2)

The backend of the system is implemented using FastAPI, a Python web framework suited for handling asynchronous requests and machine learning workloads (Tech With Tim, 2025). The backend is deployed on an Amazon EC2 instance, enabling remote access and providing a scalable environment for processing requests.

The backend performs several critical functions:

- Receiving audio files from client interfaces
- Storing and managing uploaded files temporarily
- Performing audio preprocessing using FFmpeg
- Preparing audio data for model inference
- Sending processed audio to the machine learning model
- Returning the prediction results to the client interface

FastAPI was selected due to its asynchronous capabilities and ease of integration with Python machine learning frameworks. Hosting the backend on EC2 allows the system to operate in the cloud which provides a more realistic deployment environment.

4.6 Audio Processing Pipeline

Audio preprocessing is a very important component of the system, as the machine learning models require consistent and high-quality input data. Voice notes received from WhatsApp are typically compressed and stored in .ogg format, which is not suitable for most deep learning models.

To manage this, the backend integrates FFmpeg (ffmpeg.org, n.d.), to perform several preprocessing steps, including the following:

- Conversion from .ogg to .wav format
- Normalisation of audio levels
- Resampling to a consistent frequency
- Augmentation techniques applied during training

These steps ensure that the audio data fed into the model is standardised, which improves model performance. The preprocessing pipeline was designed to closely match the

conditions used during model training so that consistency between training and inference stages could be ensured.

4.7 Machine Learning Model Deployment

Training was conducted locally but the trained model was deployed using AWS EC2 for inference.

EC2 is used solely as a hosting and inference platform in this project as well as hosting the backend, rather than for training or dataset storage. This design choice allows the model to be exposed as a scalable endpoint.

The workflow for model inference is as follows:

1. The backend prepares the pre-processed audio file
2. The audio is sent to the model
3. The model processes the input and generates a prediction score
4. The result is returned to the backend
5. The backend formats the response and sends it to the user

4.8 Training Environment

All model training and experimentation were conducted locally rather than in the cloud. This decision was primarily influenced by cost considerations and the availability of suitable local hardware.

The training pipeline included:

- Dataset preparation using publicly available and collected audio samples
- Application of data augmentation techniques
- Fine-tuning of the Wav2Vec2 model
- Evaluation using several standard performance metrics

4.9 Architectural Considerations and Design Decisions

Several key design decisions influenced the final system architecture:

- Modularity: Each component, interface, backend, and model, operates independently
- Accessibility: WhatsApp integration ensures ease of use for non-technical users
- Scalability: Cloud hosting via AWS allows for future expansion
- Flexibility: Support for multiple interfaces demonstrates extensibility
- Cost efficiency: Local training reduces reliance on expensive cloud resources

Despite these strengths, the architecture also has limitations. The reliance on WhatsApp Web instead of the official API may introduce stability issues, and the quality possible for the AI model is limited to the local compute resources which in this case were efficient for a small-scale model.

Overall, the system architecture demonstrates a practical and scalable way to integrate deep learning audio analysis into commonly used communication platforms. It balances usability, performance, and modularity, while also reflecting realistic constraints and trade-offs found during development.

5. System Implementation

5.1 Overview of Implementation Approach

Each component was implemented using technologies appropriate to its role. This section details the implementation of the system across four main components:

- Web interface
- WhatsApp chatbot
- Backend API
- Cloud deployment

For each component, key functions and design decisions are discussed in detail.

5.2 Web Interface Implementation

The web interface was developed using HyperText Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript and hosted via GitHub Pages. It acts as a lightweight client for interacting with the backend API.

5.2.1 File Handling and User Input

The interface supports both file upload and drag-and-drop functionality.

```
// Upload click
dropZone.addEventListener("click", () => fileInput.click());

fileInput.addEventListener("change", e => {
  handleFile(e.target.files[0]);
});

// Drag/drop
dropZone.addEventListener("dragover", e => {
  e.preventDefault();
  dropZone.classList.add("dragover");
});

dropZone.addEventListener("dragleave", () => {
  dropZone.classList.remove("dragover");
});

dropZone.addEventListener("drop", e => {
  e.preventDefault();
  dropZone.classList.remove("dragover");
  handleFile(e.dataTransfer.files[0]);
});

function handleFile(file) {
  if (!file) return;
  selectedFile = file;
  preview(file);
}

function preview(file) {
  const url = URL.createObjectURL(file);
  audioPreview.src = url;
  audioPreview.style.display = "block";
  status.innerText = "File ready: " + file.name;
}
```

Figure 7 File Handling for user interface

The following event listeners were implemented:

- Click: triggers hidden file input
- Change: handles file selection
- dragover, dragleave, drop: enable drag-and-drop behaviour

The `handleFile()` function stores the selected file and passes it to a preview function, so the user has immediate feedback.

5.2.2 Audio Preview and Feedback

```
function preview(file) {  
  const url = URL.createObjectURL(file);  
  audioPreview.src = url;  
  audioPreview.style.display = "block";  
  status.innerText = "File ready: " + file.name;  
}
```

Figure 8 Preview function

The preview system uses `URL.createObjectURL(file)`.

This creates a temporary local URL for the selected audio file, allowing playback without uploading it first.

5.2.3 Audio Recording Pipeline

One of the more technically significant features is in-browser audio recording.

```
// RECORDING  
async function startRecording() {  
  const stream = await navigator.mediaDevices.getUserMedia({ audio: true });  
  
  mediaRecorder = new MediaRecorder(stream);  
  recordedChunks = [];  
  
  mediaRecorder.ondataavailable = e => recordedChunks.push(e.data);  
  
  mediaRecorder.onstop = async () => {  
    const blob = new Blob(recordedChunks, { type: "audio/webm" });  
    selectedFile = new File([blob], "recording.webm");  
    preview(selectedFile);  
    status.innerText = "Recording ready";  
  };  
  
  mediaRecorder.start();  
  status.innerText = "Recording...";  
}  
  
function stopRecording() {  
  if (mediaRecorder) {  
    mediaRecorder.stop();  
  }  
}
```

Figure 9 In Browser Recording

1. Audio Capture

- `navigator.mediaDevices.getUserMedia({ audio: true })` (developer.mozilla.org, 2023)
- Captures microphone input directly from the browser

2. MediaRecorder API

- Records raw audio chunks
- Stores them in `recordedChunks`

5.2.4 API Communication

```
// Upload
async function uploadFile() {
  if (!selectedFile) {
    alert("Please select or record audio first.");
    return;
  }

  const formData = new FormData();
  formData.append("file", selectedFile);

  spinner.style.display = "block";
  status.innerText = "Analysing...";
  resultDiv.innerHTML = "";

  try {
    const response = await fetch(API_URL, {
      method: "POST",
      body: formData
    });

    const data = await response.json();

    const cls = data.verdict === "REAL" ? "real" : "fake";

    resultDiv.innerHTML = `
      <div class="${cls}">Verdict: ${data.verdict}</div>

      <div>Real: ${(data.confidence_real * 100).toFixed(1)}%</div>
      <div class="bar">
        <div class="bar-fill" style="width:${data.confidence_real*100}%; background:green;"></div>
      </div>

      <div>Fake: ${(data.confidence_fake * 100).toFixed(1)}%</div>
      <div class="bar">
        <div class="bar-fill" style="width:${data.confidence_fake*100}%; background:red;"></div>
      </div>
    `;

    status.innerText = "Done";
  } catch (err) {
    console.error(err);
    status.innerText = "Error connecting to server.";
  }
}
```

Figure 10 uploadFile

The frontend sends audio to the backend using `fetch()` API and `FormData` for file transfer.

5.3 Backend API Implementation (FastAPI)

The backend is the core processing component of the system.

5.3.1 Model Loading and Device Management

```
# Load Model
print("Loading model...")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(MODEL_DIR)
model = Wav2Vec2ForSequenceClassification.from_pretrained(MODEL_DIR)

model = model.to(device)
model.eval()

if device.type == "cuda" and next(model.parameters()).dtype == torch.bfloat16:
    model = model.bfloat16()

print(f"Model loaded on {device}")

print("Label config:")
print(model.config.id2label)

LABELS_ARE_SWAPPED = False

if model.config.id2label == {0: "LABEL_0", 1: "LABEL_1"}:
    print("Warning: generic labels detected")
elif "spoof" in str(model.config.id2label.get(0, "")):
    LABELS_ARE_SWAPPED = True

print("System ready\n")

# Health Check
@app.get("/")
def health():
    return {
        "status": "running",
        "model": "wav2vec2-deepfake-detector"
    }
```

Figure 11 Model Loading

Key functions:

- `torch.device()`: selects Central Processing Unit (CPU) or Graphics Processing Unit (GPU)
- `from_pretrained()`: loads trained model and feature extractor
- `model.eval()`: disables training behaviour (e.g. dropout)
- Brain Floating Point format (bfloat16) support cuda support used when available

5.3.2 Windowing Segmentation

```
# Windowing Function
def create_windows(audio, window_size, stride):
    if len(audio) <= window_size:
        return [np.pad(audio, (0, window_size - len(audio)))]

    windows = []

    for start in range(0, len(audio) - window_size + 1, stride):
        windows.append(audio[start:start + window_size])

    # ensure last window is included
    last_start = len(audio) - window_size
    if windows[-1].shape[0] != window_size:
        windows.append(audio[last_start:last_start + window_size])

    return windows
```

Figure 12 Windowing Function

The function: `create_windows(audio, window_size, stride)`

- Splits long audio into overlapping fixed length segments (Thor, 2026)
- Ensures all inputs match the model's expected input size
- Adds padding for short audio clips
- Ensures last segment is not discarded

This approach was introduced as voice messages vary in length and single pass inference is unreliable for long audio.

5.3.3 API Endpoint Design

```
# Main Route
@app.post("/detect")
async def detect(file: UploadFile = File(...)):

    job_id = str(uuid.uuid4())

    input_path = f"temp_input_{job_id}.ogg"
    output_path = f"temp_output_{job_id}.wav"

    try:
        # Save upload
        with open(input_path, "wb") as f:
            f.write(await file.read())
```

Each request:

- Accepts an uploaded audio file

- Processes it through the inference pipeline
- Returns a structured JavaScript Object Notation (JSON) response containing prediction results

To ensure request isolation, each upload is assigned a unique identifier. This prevents file name collisions and supports concurrent requests.

5.3.4 File Handling and Audio Conversion

Uploaded files are temporarily saved before processing. The system then uses FFmpeg to standardise audio format. While voice isolation and background noise cancellation could be introduced, these options tend to add extra artifacts, so this audio conversion aims just to normalize audio and preserve as much of the original as possible.

```
# Convert audio
result = subprocess.run([
    "ffmpeg", "-y",
    "-i", input_path,
    "-ar", str(SAMPLE_RATE),
    "-ac", "1",
    "-sample_fmt", "s16",
    output_path
], capture_output=True, text=True)

if result.returncode != 0:
    return {
        "verdict": "ERROR",
        "error": "FFmpeg conversion failed",
        "confidence_real": 0.0,
        "confidence_fake": 0.0
    }
```

Figure 13 FFmpeg Conversion

What this does:

- Converts .ogg to .wav
- Resamples audio to 16kHz
- Forces mono channel input
- 16-bit Pulse-Code Modulation (PCM) format

Why FFmpeg is used:

- Industry standard for audio processing
- Ensures consistent model input regardless of source device or format

If conversion fails, the system returns a structured error response to prevent crashes.

5.3.5 Audio Augmentation

```
# Load Audio
audio, _ = sf.read(output_path, dtype="float32")

print(f"Audio loaded: {len(audio)} samples")

# Create Windows
windows = create_windows(audio, WINDOW_SIZE, STRIDE)

print(f"Processing {len(windows)} windows")

all_probs = []

# Inference per window
for window in windows:

    # Normalize per window
    peak = np.abs(window).max()
    if peak > 0:
        window = window / peak
```

Figure 14 Audio Augmentations

Before feature extraction, each window is individually normalised which ensures:

- Consistent amplitude scaling across all segments
- Removal of volume bias
- Model focuses on spectral features rather than loudness

Normalisation per window is important because different parts of speech may have different intensities and deepfake artifacts are not amplitude dependent.

5.3.6 Feature Extraction

```
inputs = feature_extractor(
    window,
    sampling_rate=SAMPLE_RATE,
    return_tensors="pt",
    padding=False
)
```

Figure 15 Feature Extractor

This converts raw waveform into tensor format. It prepares input for the Wav2Vec2 model.

5.3.7 Model Inference and Decision Logic

```
input_values = inputs["input_values"].to(device)

if next(model.parameters()).dtype == torch.bfloat16:
    input_values = input_values.bfloat16()

with torch.no_grad():
    outputs = model(input_values=input_values)
    logits = outputs.logits[0].float()
    probs = torch.softmax(logits, dim=-1)

all_probs.append(probs.cpu().numpy())

# Average Voting
all_probs = np.array(all_probs)
avg_probs = np.mean(all_probs, axis=0)

# Label Handling
if LABELS_ARE_SWAPPED:
    confidence_real = float(avg_probs[1])
    confidence_fake = float(avg_probs[0])
else:
    confidence_real = float(avg_probs[0])
    confidence_fake = float(avg_probs[1])

verdict = "FAKE" if confidence_fake >= FAKE_THRESHOLD else "REAL"

print(f"Verdict: {verdict}")

return {
    "verdict": verdict,
    "confidence_real": round(confidence_real, 3),
    "confidence_fake": round(confidence_fake, 3),
    "windows_processed": len(windows)
}
```

Figure 16 Model Inference

Each window is passed through the model individually to make prediction probabilities. These probabilities are then aggregated using an average voting method, which helps to reduce the impact of noisy segments.

The system includes label handling to correct any inconsistencies in model output ordering although this was mostly only an issue during development.

Final decision: If fake confidence greater than 0.5 then FAKE Else REAL.

The API returns the final verdict, confidence scores and the number of windows processed.

5.4 WhatsApp Bot Implementation

The WhatsApp bot acts as the primary user interface.

5.4.1 Authentication and Session Handling

```
// bot.js
import pkg from 'whatsapp-web.js'
const { Client, LocalAuth } = pkg
import qrcode from 'qrcode-terminal'
import axios from 'axios'
import FormData from 'form-data'
import fs from 'fs'

const client = new Client({
  authStrategy: new LocalAuth(),
  puppeteer: {
    headless: true,
    args: [
      '--no-sandbox',
      '--disable-setuid-sandbox',
      '--disable-dev-shm-usage',
      '--disable-accelerated-2d-canvas',
      '--no-first-run',
      '--no-zygote',
      '--disable-gpu'
    ]
  }
})

client.on('qr', (qr) => {
  console.log('\nQR Code generated. Scan with WhatsApp:\n')
  qrcode.generate(qr, { small: true })
  console.log('\nOr open WhatsApp and scan the QR above\n')
})
```

Figure 17 Setup and QR Code Generation

Puppeteer is configured to run in headless mode, allowing the bot to operate without opening a visible browser window. The other arguments such as `--no-sandbox` and `--disable-dev-shm-usage` are included to improve stability and ensure compatibility when running in certain environments.

- `client.on('qr')`: listens for login request
- `qrcode.generate()`: prints QR code to terminal

When the bot starts without an existing session, a QR code is generated and displayed in the terminal. This has to be scanned using the WhatsApp mobile app to authenticate the session. For the purpose of this project, a sim I have control of is acting as the WhatsApp account for the bot.

5.4.2 Event-Driven Message Handling

```
client.on('ready', () => {
  console.log('Bot is ready and connected to WhatsApp!')
  console.log('Listening for voice messages...\n')
})

client.on('authenticated', () => {
  console.log('Authentication successful')
})

client.on('auth_failure', () => {
  console.error('Authentication failed. Delete .webjs_auth folder and try again.')
})

client.on('disconnected', (reason) => {
  console.log('Client disconnected:', reason)
})

// Log all incoming messages
client.on('message', async (message) => {
  const timestamp = new Date().toLocaleTimeString('en-IE', { hour12: false })
  const contact = await message.getContact()
  const senderName = contact.pushname || contact.number

  // Log every message received
  console.log(`[${timestamp}] Message from ${senderName}: ${message.type}`)

  // Handle both original voice messages ptt and regular audio messages
  const isVoiceMessage = message.hasMedia && (message.type === 'ptt' || message.type === 'audio')
})
```

Figure 18 Event-Driven Handling

The bot uses event driven listeners to manage itself and handle incoming data. All incoming messages are logged with timestamps and sender information.

Message filtering is applied to ensure only relevant data is processed. The system checks whether a message contains media and verifies that it is a voice message (push-to-talk or audio). This prevents any unnecessary processing of text or unsupported message types and ensures the pipeline only handles audio inputs.

5.4.3 API Communication

```
try {
  // Send acknowledgement
  await message.reply('Received your voice message. Analysing for deepfake detection...')

  console.log('Downloading audio...')
  const media = await message.downloadMedia()
  const buffer = Buffer.from(media.data, 'base64')

  const form = new FormData()
  form.append('file', buffer, {
    filename: 'audio.ogg',
    contentType: 'audio/ogg'
  })

  console.log('Sending to deepfake detection model...')
  const startTime = Date.now()

  const res = await axios.post('http://127.0.0.1:8000/detect', form, {
    headers: form.getHeaders(),
    timeout: 30000
  })

  const processingTime = ((Date.now() - startTime) / 1000).toFixed(2)
  const { verdict, confidence_real, confidence_fake } = res.data
}
```

Figure 19 Bot to API Communication

When a valid voice message is received, the bot first sends an acknowledgement to tell the user that processing has started.

The audio is then downloaded using `message.downloadMedia()`, which returns base64-encoded data. This is converted into a binary buffer using `Buffer.from()`, allowing it to be handled as a file.

The audio is packaged into a `FormData` object to ensure it is compatible with the FastAPI backend. The request is sent using `axios.post()` to the `/detect` endpoint, with a timeout to prevent the system from hanging on slow or failed requests.

Processing time is recorded to measure system performance, and the response from the API contains the predicted verdict and confidence scores.

5.4.4 Response Formatting

```
// Format response message
const resultMessage =
  `Deepfake Detection Result\n\n` +
  `Verdict: ${verdict}\n` +
  `Confidence (Real): ${((confidence_real * 100).toFixed(1))}%\n` +
  `Confidence (Fake): ${((confidence_fake * 100).toFixed(1))}%\n\n` +
  `Processed in ${processingTime}s`

await message.reply(resultMessage)
console.log('Result sent to user\n')

} catch (err) {
  console.error('ERROR:', err.message)
  if (err.response) {
    console.error(' API Response:', err.response.data)
  }
  await message.reply('Sorry, deepfake detection failed. Please try again.')
}
})
```

Figure 20 User Response Formatting

After receiving the prediction from the backend, the bot formats the output into user-friendly message. This includes the final verdict, confidence scores and total processing time. The formatted message is sent back to the user using `message.reply`.

Error handling is implemented using a try/catch block. If any failure occurs the system logs the issue and returns a failure message to the user.

5.5 Cloud Implementation

The system was deployed to a cloud environment using an Amazon EC2 instance running Ubuntu. The EC2 instance was configured to host both the FastAPI inference API and the WhatsApp bot. This approach allowed the model to be accessible remotely in real time.

The project codebase was transferred to the EC2 instance using a Git workflow, where the repository was cloned directly onto the server. All required dependencies for the API were installed within a Python virtual environment, including PyTorch, Hugging Face Transformers, FastAPI, and all audio processing libraries such as soundfile and ffmpeg. System dependencies were also installed on the instance to support audio decoding and preprocessing.

The deepfake detection model was stored locally on the EC2 instance within the /model directory, containing the trained Wav2Vec2 classification weights, configuration files, and preprocessor settings.

The FastAPI application was executed using Uvicorn, exposed on port 8000, and bound to 0.0.0.0 to allow external access. This enabled the WhatsApp bot service to communicate with the model via HTTP requests. The WhatsApp bot, was also hosted on the same EC2 instance and acted as the interface layer between users and the inference API. Voice messages received through WhatsApp were automatically downloaded, converted into a suitable audio format using FFmpeg, and forwarded to the API for classification, then deleted to avoid unnecessary storage.

To verify correct deployment, testing was performed by sending WhatsApp voice messages to the bot. The system successfully processed incoming audio, performed inference using the model, and returned a classification result indicating whether the audio was real or synthetic.

Name and tags

Name

deepfake-whatsapp-bot

Add additional tags

Application and OS Images (Amazon Machine Image)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE Linux

Debian

Debian

Browse more AMIs

Including AMIs from AWS Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type

ami-0d6f64d6a3b2020a8 (64-bit (x86)) / ami-0a0f534ab418cab0f (64-bit (ARM))

Free tier eligible

Virtualization: hvm

ENX enabled: true

Root device type: ebs

Description

Ubuntu Server 26.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Instance type

t3.small

Family: t3

2 vCPU

2 GiB Memory

Current generation: true

On-Demand Windows base pricing: 0.0412 USD per Hour

Free tier eligible

On-Demand Linux base pricing: 0.0228 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0263 USD per Hour

On-Demand SUSE base pricing: 0.0538 USD per Hour

On-Demand RHEL base pricing: 0.0516 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

Key pair (login)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

Deepfake-Bot-Key

Create new key pair

Description - required

Security Group for FYP

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 0.0.0.0/0)

Remove

Type

ssh

Protocol

TCP

Port range

22

Source type

Anywhere

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Security group rule 2 (TCP, 443, 0.0.0.0/0)

Remove

Type

HTTPS

Protocol

TCP

Port range

443

Source type

Anywhere

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Security group rule 3 (TCP, 8000)

Remove

Type

Custom TCP

Protocol

TCP

Port range

8000

Source type

Custom

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Figure 21 EC2 Configuration

34

```

ubuntu@ip-172-31-34-235:~/ChrisKelleher1947.github.io$ ls
FYP-Diagram.drawio.png  ProjectPlanTimeline.md  _config.yml          bot                  test-scripts
Model-Training-Files    README.md               aiTrainingFlow.png   demo.html           venv
ProjectArchitecture.md  TechnologiesUsed.md     api                  package-lock.json
ubuntu@ip-172-31-34-235:~/ChrisKelleher1947.github.io$ cd bot
ubuntu@ip-172-31-34-235:~/ChrisKelleher1947.github.io/bot$ node bot.js

WhatsApp Deepfake Detection Bot

Starting bot...

Authentication successful
Bot is ready and connected to WhatsApp!
Listening for voice messages...

```

Figure 22 WhatsApp Bot running on EC2

```

(venv) ubuntu@ip-172-31-34-235:~/ChrisKelleher1947.github.io$ cd api
(venv) ubuntu@ip-172-31-34-235:~/ChrisKelleher1947.github.io/api$ uvicorn main:app --host 0.0.0.0 --port 8000

Loading model...
Loading weights: 100% | 215/215 [00:00<00:00, 1274.50it/s]
Model loaded on cpu
System ready

INFO: Started server process [6438]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Audio loaded: 69440 samples
Processing 1 windows
Verdict: FAKE
INFO: 127.0.0.1:35124 - "POST /detect HTTP/1.1" 200 OK
Audio loaded: 28480 samples
Processing 1 windows
Verdict: FAKE
INFO: 127.0.0.1:48612 - "POST /detect HTTP/1.1" 200 OK

```

Figure 23 Model Loaded on EC2

5.6 Ngrok for Development

```

ngrok                                     (Ctrl+C to quit)

* One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             ckelleher1312@gmail.com (Plan: Free)
Update              update available (version 3.38.0, Ctrl-U to update)
Version             3.36.1-msix-stable
Region              Europe (eu)
Latency             87ms

```

Figure 24 ngrok tunnel output

```

HTTP Requests
-----
14:33:56.844 BST POST /detect 200 OK
14:33:31.882 BST POST /detect 200 OK
14:32:28.143 BST POST /detect 200 OK
14:31:45.261 BST POST /detect 200 OK
14:31:00.742 BST POST /detect 200 OK
14:29:44.254 BST POST /detect 200 OK
14:14:57.931 BST POST /detect 200 OK

```

Figure 25 ngrok http requests

Used to Expose local backend to internet and allow GitHub Pages frontend to communicate (Mintlify, 2026).

6. Model Training and Development

This section outlines the development and training of the deep learning models used for synthetic speech detection. The training process was carried out in multiple stages, with different notebook experiments used to evaluate model architectures and performance improvements. The final model to be used was the Wav2Vec model due to superior performance. Much of the training notebooks share the same code and process, so for this reason Wav2Vec will be fully covered while the Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) models will be covered only in what differed in their training notebooks.

6.1 Data Exploration for ASVspoof

Before model training, an exploration of the dataset was carried out to understand its structure, class distribution, and visualize some audio characteristics. This step was essential to ensure the model was trained on balanced and representative data.

The primary dataset used was the ASVspoof 2019 Logical Access (LA) dataset (Yamagishi et al., 2021), which is widely used as a benchmark for synthetic speech detection research.

6.1.1 Dataset Structure and Label Analysis

The dataset includes a protocol file that maps each audio clip to its corresponding label and metadata.

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# This file says which audio clips are real and which are fake
protocol_path = r"C:\deepfake-project\data\asvspoof\ASVspoof2019_LA_cm_protocols\ASVspoof2019_LA_cm_train.trn.txt"

# Column names taken from the ASVspoof readme
df = pd.read_csv(protocol_path, sep=" ", header=None,
                 names=["speaker", "file_id", "env", "attack", "label"])

print(df.head(10))
print(f"Total files: {len(df)}")
print(f"Bonafide (real): {(df['label'] == 'bonafide').sum()}")
print(f"Spooof (fake): {(df['label'] == 'spooof').sum()}")

speaker  file_id env attack  label
0 LA_0079 LA_T_1138215 - - bonafide
1 LA_0079 LA_T_1271820 - - bonafide
2 LA_0079 LA_T_1272637 - - bonafide
3 LA_0079 LA_T_1276960 - - bonafide
4 LA_0079 LA_T_1341447 - - bonafide
5 LA_0079 LA_T_1363611 - - bonafide
6 LA_0079 LA_T_1596451 - - bonafide
7 LA_0079 LA_T_1608170 - - bonafide
8 LA_0079 LA_T_1684951 - - bonafide
9 LA_0079 LA_T_1699881 - - bonafide

Total files: 25380
Bonafide (real): 2580
Spooof (fake): 22800
```

Figure 26 Dataset and Labels

The dataset contains the following key fields:

- Speaker ID
- File ID
- Attack type
- Label

The dataset is heavily structured and pre-labelled. It includes both genuine or Bonafide and synthetic spoof audio. Spoof samples are generated using multiple attack algorithms.

The class distribution was examined as well to be aware of potential imbalance between real and fake samples. This step is important because class imbalance can bias the model towards the majority class if not properly managed during training.

6.1.2 Loading and Inspecting Audio Samples

To better understand the dataset, individual audio samples from both classes were loaded and visualised.

```
import os

# Load one bonafide file
real_file = r"C:\deepfake-project\data\asvspoof\ASVspoof2019_LA_train\flac\LA_T_1138215.flac"
real_audio, sr = librosa.load(real_file, sr=16000)

# Get the first spoof file from the dataframe
spoof_row = df[df['label'] == 'spoof'].iloc[0]
spoof_file = rf"C:\deepfake-project\data\asvspoof\ASVspoof2019_LA_train\flac\{spoof_row['file_id']}.flac"
spoof_audio, sr = librosa.load(spoof_file, sr=16000)

# Plot both waveforms side by side
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].plot(real_audio)
axes[0].set_title("Bonafide (Real)")
axes[0].set_xlabel("Sample")
axes[0].set_ylabel("Amplitude")

axes[1].plot(spoof_audio)
axes[1].set_title(f"Spoof (Fake) - {spoof_row['attack']}")
axes[1].set_xlabel("Sample")
axes[1].set_ylabel("Amplitude")

plt.tight_layout()
plt.show()

print(f"Real audio - duration: {len(real_audio)/sr:.2f}s, samples: {len(real_audio)}")
print(f"Fake audio - duration: {len(spoof_audio)/sr:.2f}s, samples: {len(spoof_audio)}")
```

Figure 27 Visualizing Audio Samples

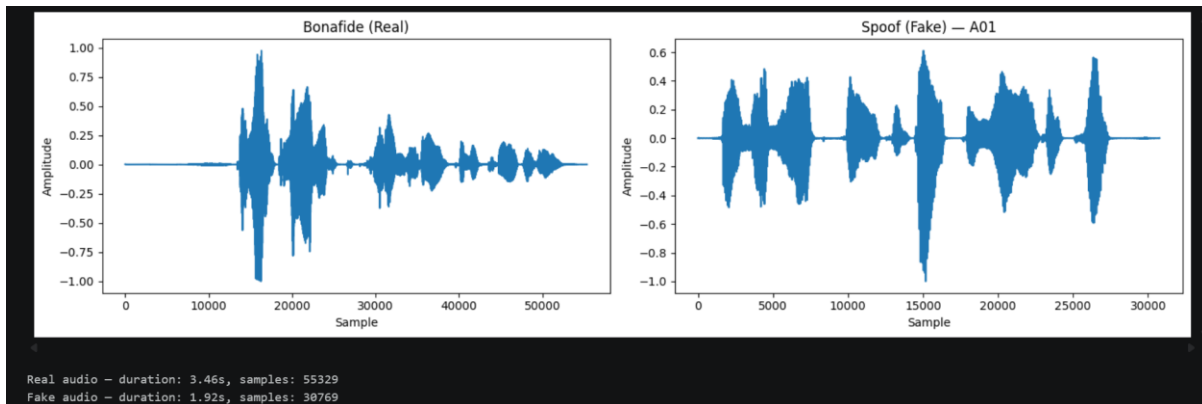


Figure 28 Real vs Fake waveform

Key observations from waveform analysis:

- Bonafide audio shows more natural amplitude variation and tends to be less stable
- Spoofed audio often appears smoother or more uniform
- Differences are subtle and not always visually obvious

This reinforces the need for deep learning models rather than a rule-based detection system, as visual cues alone is not enough to distinguish between real and fake.

6.1.3 Spectral Analysis using Log-Mel Spectrograms

To get deeper insight into frequency differences, log-mel spectrograms were generated.

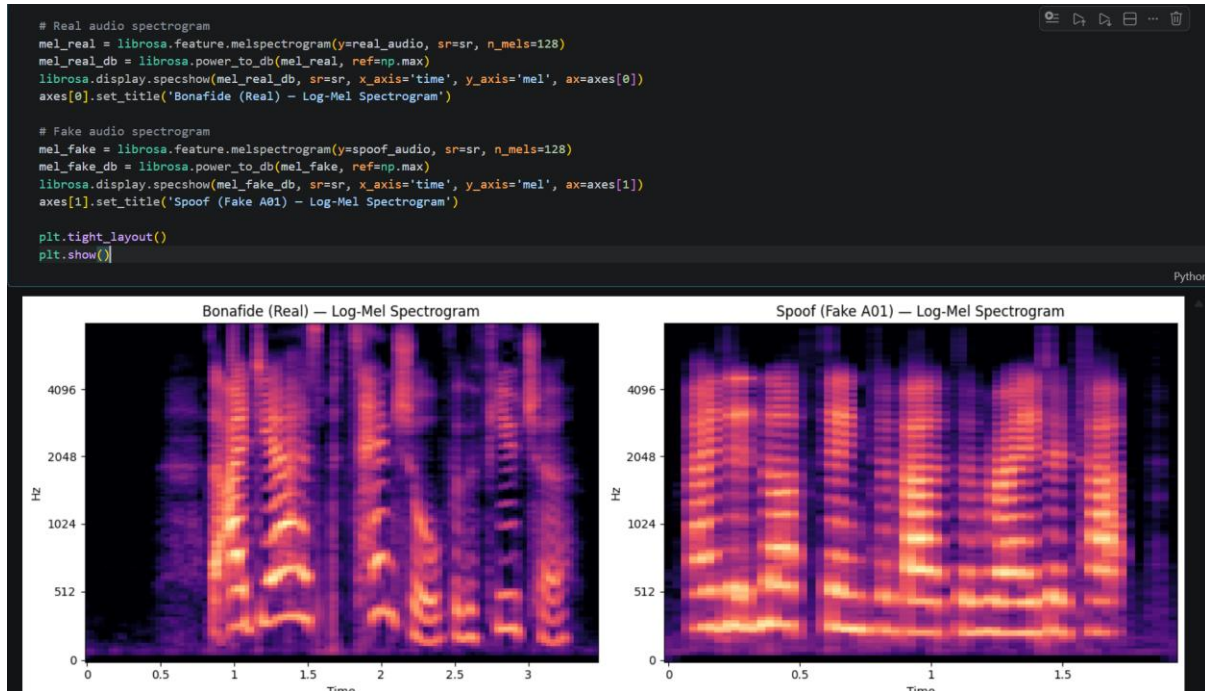


Figure 29 Log-Mel Real vs Fake Visualized

Spectrogram analysis revealed:

- Real speech contains more natural frequency variation
- Synthetic speech may show artefacts or unnatural patterns, tends to be too smooth and evenly spaced
- Differences are more visible in frequency domain than waveform domain

6.1.4 Summary of Dataset Exploration

The dataset exploration phase confirmed several important points:

- The ASVspoof dataset provides a strong benchmark for synthetic speech detection
- Real vs fake differences are not visually obvious in waveforms alone
- Audio standardisation is essential due to variability in sample length

Overall, this phase established the foundation for model training by ensuring that the dataset was well understood, properly balanced, and suitable for deep learning classification.

6.2 Wav2Vec2 Training Pipeline

This section introduces the setup of the Wav2Vec2 training pipeline.

6.2.1 Environment Setup, Imports and GPU Verification

Compute Unified Device Architecture (CUDA) environment variables are configured to improve debugging capability. Following this, all required libraries are imported. PyTorch is used as the core deep learning framework, while additional libraries such as NumPy, Pandas, SoundFile, and TorchAudio handle mathematical operations and audio processing. The Hugging Face Transformers library contains the pretrained Wav2Vec2 model and feature extractor, which are the foundation of the system.

Finally, the system verifies GPU availability. This step ensures that training will run on CUDA enabled hardware rather than CPU, which would significantly increase computation time.

```
# Cell 1: Environment setup
import os
os.environ["CUDA_LAUNCH_BLOCKING"] = "1"
os.environ["TORCH_USE_CUDA_DSA"] = "1"

# Cell 2: Imports
import os
import torch
import numpy as np
import pandas as pd
import soundfile as sf
import torchaudio
import random
from torch.utils.data import Dataset, DataLoader
from transformers import Wav2Vec2ForSequenceClassification, Wav2Vec2FeatureExtractor

# Cell 3: Check GPU
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"GPU: {torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'None'}")

PyTorch version: 2.12.0.dev20260312+cu128
CUDA available: True
GPU: NVIDIA GeForce RTX 5070 Laptop GPU
```

Figure 30 GPU config and Imports

6.2.2 Dataset Loading and Multi-Source Integration

This stage focuses on loading and combining all datasets used for training the model. The primary dataset is ASVspooF 2019 Logical Access, which is a widely used benchmark for synthetic speech detection. It provides structured protocol files that map each audio sample to its corresponding label, speaker ID, and attack type.

The protocol files are read into Pandas dataframes, and labels are converted into binary format, where bonafide speech is assigned label 0 and spoofed speech is assigned label 1. Each audio file path is then constructed based on the dataset directory structure.

```
# Paths
ASVSPOOF_ROOT = os.environ.get("ASVSPOOF_ROOT", r"C:\deepfake-project\data\asvspoof")
PROTOCOL_DIR = os.path.join(ASVSPOOF_ROOT, "ASVspooF2019_LA_cm_protocols")
TRAIN_AUDIO = os.path.join(ASVSPOOF_ROOT, "ASVspooF2019_LA_train", "flac")
DEV_AUDIO = os.path.join(ASVSPOOF_ROOT, "ASVspooF2019_LA_dev", "flac")

# Path to collected WhatsApp voice notes
WHATSAPP_VOICE_DIR = r"C:\whatsapp-voice-bot\Chriskelleher1947.github.io\bot\collected_voice_notes"

# Verify paths exist
for path_name, path in [(("ASVSPOOF_ROOT", ASVSPOOF_ROOT), ("PROTOCOL_DIR", PROTOCOL_DIR),
                        ("TRAIN_AUDIO", TRAIN_AUDIO), ("DEV_AUDIO", DEV_AUDIO))]:
    if not os.path.exists(path):
        raise FileNotFoundError(f"{path_name} does not exist: {path}")

if not os.path.exists(WHATSAPP_VOICE_DIR):
    print(f"WARNING: WhatsApp voice directory not found: {WHATSAPP_VOICE_DIR}")
    print("Proceeding with ASVspooF data only")

# Read train and dev protocol files
train_df = pd.read_csv(
    os.path.join(PROTOCOL_DIR, "ASVspooF2019_LA_cm_train.trn.txt"),
    sep=" ", header=None,
    names=["speaker", "file_id", "env", "attack", "label"]
)

dev_df = pd.read_csv(
    os.path.join(PROTOCOL_DIR, "ASVspooF2019_LA_cm_dev.trn.txt"),
    sep=" ", header=None,
    names=["speaker", "file_id", "env", "attack", "label"]
)

# Convert labels to integers - 0 = bonafide (REAL), 1 = spoof (FAKE)
train_df["label"] = train_df["label"].map({"bonafide": 0, "spoof": 1})
dev_df["label"] = dev_df["label"].map({"bonafide": 0, "spoof": 1})

# Add the full file path for each audio file
train_df["path"] = train_df["file_id"].apply(lambda x: os.path.join(TRAIN_AUDIO, f"{x}.flac"))
dev_df["path"] = dev_df["file_id"].apply(lambda x: os.path.join(DEV_AUDIO, f"{x}.flac"))
```

Figure 31 Loading ASVspooF data and assigning labels to integers

In addition to the ASVspooF dataset, real WhatsApp voice notes are also integrated into the training set that were created by myself. These are explicitly labelled as bonafide and added to the dataset to improve robustness. The inclusion of this data is important because ASVspooF samples are relatively clean and studio quality, whereas WhatsApp recordings contain compression artefacts, background noise, and differing recording quality. This helps bridge the gap between controlled datasets and real deployment conditions.

```

# Add WhatsApp voice notes if directory exists
if os.path.exists(WHATSAPP_VOICE_DIR):
    whatsapp_files = [f for f in os.listdir(WHATSAPP_VOICE_DIR) if f.endswith('.ogg')]

    whatsapp_df = pd.DataFrame({
        "speaker": ["chris"] * len(whatsapp_files),
        "file_id": [os.path.splitext(f)[0] for f in whatsapp_files],
        "env": ["whatsapp"] * len(whatsapp_files),
        "attack": ["-"] * len(whatsapp_files),
        "label": [0] * len(whatsapp_files), # All are bonafide (REAL)
        "path": [os.path.join(WHATSAPP_VOICE_DIR, f) for f in whatsapp_files]
    })

# Merge with training data
train_df_original_count = len(train_df)
train_df = pd.concat([train_df, whatsapp_df], ignore_index=True)

print(f"Added {len(whatsapp_df)} WhatsApp voice notes to training set")
print(f"ASVspoof samples: {train_df_original_count}")
print(f"WhatsApp samples: {len(whatsapp_df)}")
print(f"Total: {len(train_df)}")
else:
    print("No WhatsApp voice notes added (directory not found)")

print(f"Final dataset:")
print(f"Training samples: {len(train_df)}")
print(f"Dev samples: {len(dev_df)}")
print(f"Train label split: {train_df['label'].value_counts().to_dict()}")
print(f"Dev label split: {dev_df['label'].value_counts().to_dict()}")

```

Figure 32 Merging WhatsApp Voice Notes

After merging both datasets, the final class distribution is printed to ensure that the training data remains balanced and suitable for classification training.

```

Added 53 WhatsApp voice notes to training set
ASVspoof samples: 25380
WhatsApp samples: 53
Total: 25433

Final dataset:
Training samples: 25433
Dev samples: 24844
Train label split: {1: 22800, 0: 2633}
Dev label split: {1: 22296, 0: 2548}

```

Figure 33 Dataset Loading Output

6.2.3 Audio Loading, Normalisation and Feature Preparation

This section describes the implementation of the custom PyTorch dataset class used to prepare audio samples for training the Wav2Vec2 model. The dataset pipeline is responsible for loading raw audio, applying augmentation, standardising input length, normalising amplitude, and converting waveform data into a suitable format.

The dataset class first handles audio loading through a unified interface that supports both standard Waveform Audio File Format (WAV)/ Free Lossless Audio Codec (FLAC) files and WhatsApp OGG voice notes. WhatsApp audio is not natively compatible with most deep learning pipelines, so FFmpeg is used to convert OGG files into a standard WAV format.

```

class ASVspoofDataset(Dataset):
    def __init__(self, df, feature_extractor, augment=True):
        self.df = df
        self.feature_extractor = feature_extractor
        self.augment = augment

        # augmentation probabilities
        self.probs = {
            "opus": 0.8,
            "noise": 0.5,
            "bandpass": 0.5,
            "volume": 0.4,
            "pitch": 0.3
        }

    def load_audio(self, path):
        # WhatsApp .ogg handling via ffmpeg
        if path.endswith(".ogg"):
            tmp_wav = tempfile.mktemp(suffix=".wav")

            try:
                subprocess.run([
                    "ffmpeg", "-y", "-loglevel", "error",
                    "-i", path,
                    "-ar", str(SAMPLE_RATE),
                    "-ac", "1",
                    tmp_wav
                ], check=True)

                audio, _ = sf.read(tmp_wav, dtype="float32")
                return audio

            finally:
                if os.path.exists(tmp_wav):
                    os.remove(tmp_wav)

        # normal audio

```

Figure 34 Audio Loading

This conversion ensures that all audio is resampled to 16kHz and converted to mono channel input. These settings are consistent with the preprocessing requirements of Wav2Vec2, which expects single-channel waveform input at a fixed sampling rate.

Temporary files are used during conversion to avoid memory issues, and are automatically deleted after processing.

A major component of this dataset implementation is the audio augmentation pipeline, which is designed to simulate real distortions and improve model generalisation. Rather than relying on clean, studio-quality audio, the model is intentionally exposed to variations that are similar to a realistic environment.

The augmentation process is probabilistic, meaning that each transformation is applied randomly based on predefined probabilities. This ensures that the model does not overfit to a single augmentation pattern.

The augmentation techniques include compression simulation, noise injection, volume scaling, pitch shifting, and frequency filtering. Each method targets a different type of distortion commonly found in real-world audio, particularly voice messages transmitted over mobile networks and messaging platforms (Joshi, 2024).

One of the most important augmentations is Opus codec simulation (Mahbub, 2025), which replicates the compression artefacts introduced by WhatsApp voice messages. This is done by encoding the audio into the Opus format using FFmpeg and then decoding it back into waveform form.

```
def augment_audio(self, audio):  
    # Opus simulation via ffmpeg  
    if random.random() < self.probs["opus"]:  
        tmp_in = tempfile.mktemp(suffix=".wav")  
        tmp_ogg = tempfile.mktemp(suffix=".ogg")  
        tmp_out = tempfile.mktemp(suffix=".wav")  
  
        try:  
            sf.write(tmp_in, audio, SAMPLE_RATE)  
  
            bitrate = random.choice([12, 16, 24])  
  
            subprocess.run([  
                "ffmpeg", "-y", "-loglevel", "error",  
                "-i", tmp_in,  
                "-c:a", "libopus",  
                "-b:a", f"{bitrate}k",  
                tmp_ogg  
            ], check=True)  
  
            subprocess.run([  
                "ffmpeg", "-y", "-loglevel", "error",  
                "-i", tmp_ogg,  
                "-ar", str(SAMPLE_RATE),  
                "-ac", "1",  
                tmp_out  
            ], check=True)  
  
            audio, _ = sf.read(tmp_out, dtype="float32")  
  
        finally:  
            for f in [tmp_in, tmp_ogg, tmp_out]:  
                if os.path.exists(f):  
                    os.remove(f)
```

Figure 35 Opus Compression

The bitrate is randomly selected within a predefined range to simulate different network conditions and compression levels. This step is particularly important because deepfake detection systems trained only on clean audio would fail when exposed to heavily compressed samples.

Beyond compression simulation, several signal-level augmentations are applied directly in the waveform domain.

Additive noise is introduced to simulate environmental background sounds, which is especially relevant for mobile recordings. Volume scaling is used to replicate differences in recording distance and microphone sensitivity, while pitch shifting introduces slight variations in vocal tone to improve model robustness.

A bandpass filter is also applied in some cases to remove extreme frequency components and simulate bandwidth limited transmission channels. This helps the model focus on the most informative speech frequency ranges rather than noise artefacts (docs.scipy.org, n.d.).

These augmentations are applied independently and probabilistically, ensuring that each training sample can exhibit a unique combination of distortions.

```

def add_noise(self, audio):
    noise = np.random.randn(len(audio)) * random.uniform(0.003, 0.02)
    return audio + noise

def change_volume(self, audio):
    return audio * random.uniform(0.5, 1.5)

def pitch_shift(self, audio, semitones):
    if semitones == 0:
        return audio
    rate = 2 ** (semitones / 12)
    idx = np.round(np.arange(0, len(audio), rate)).astype(int)
    idx = idx[idx < len(audio)]
    return audio[idx]

def bandpass_filter(self, audio, lowcut=50, highcut=400):
    nyq = SAMPLE_RATE / 2
    sos = signal.butter(
        5,
        [lowcut / nyq, highcut / nyq],
        btype="band",
        output="sos"
    )
    return signal.sosfilt(sos, audio).astype(np.float32)

```

Figure 36 Augmentation Functions

To ensure compatibility with batch training, all audio samples are constrained to a fixed duration of 4 seconds. Longer audio clips are truncated, while shorter clips are padded with zeros. This standardisation is necessary because transformer architectures require fixed size input tensors for efficient batch processing.

```

def fix_length(self, audio):
    if len(audio) > MAX_SAMPLES:
        audio = audio[:MAX_SAMPLES]
    else:
        audio = np.pad(audio, (0, MAX_SAMPLES - len(audio)))
    return audio

```

Figure 37 Fixed Length Function

Following this, amplitude normalisation is applied. Each audio sample is scaled based on its peak value to ensure consistent signal amplitude across the dataset. This prevents the model from learning irrelevant features such as recording volume and instead encourages it to focus on the characteristics of speech.

Once preprocessing is complete, the audio waveform is passed into the Wav2Vec2 feature extractor. Unlike traditional speech processing pipelines that rely on handcrafted features such as Mel-Frequency Cepstral Coefficients (MFCCs) or spectrograms, Wav2Vec2 operates directly on raw waveform input.

The feature extractor converts the audio into a tensor representation that is compatible with the transformer encoder. This allows the model to learn speech representations directly from raw signal data, which has been shown to improve performance in speech classification tasks.

Finally, both the processed input tensor and the corresponding label are returned. The input tensor is converted into bfloat16 format.

```
def normalize(self, audio):
    peak = np.abs(audio).max()
    return audio / peak if peak > 0 else audio

def __getitem__(self, idx):
    row = self.df.iloc[idx]
    audio = self.load_audio(row["path"])

    if self.augment and not row["path"].endswith(".ogg"):
        audio = self.augment_audio(audio)

    audio = self.fix_length(audio)
    audio = self.normalize(audio)

    inputs = self.feature_extractor(
        audio,
        sampling_rate=SAMPLE_RATE,
        return_tensors="pt"
    )

    return {
        "input_values": inputs["input_values"].squeeze(0).to(torch.bfloat16),
        "label": torch.tensor(row["label"], dtype=torch.long)
    }
```

Figure 38 Feature Extraction and Normalisation

6.2.4 Model Loading and Architecture Configuration

This stage involves loading the pretrained Wav2Vec2 model and adapting it for the specific task of classification between bonafide and spoofed speech (Baevski et al., 2020).

The feature extractor and model are both loaded from the Hugging Face Transformers library. The model is then reconfigured to output two classification labels corresponding to real and synthetic speech.

To optimise training efficiency and reduce overfitting, a partial fine-tuning strategy is applied. The convolutional feature encoder layers, which learn low level acoustic representations, are frozen to preserve pretrained speech features. The lower six transformer encoder layers are also frozen, reducing the number of trainable parameters while still allowing the upper layers to adapt to the deepfake detection task (Stafford, 2025).

The model is then transferred to GPU and converted to bfloat16 precision where supported, significantly reducing memory usage and improving training.

Finally, a parameter summary is generated to confirm the proportion of trainable versus frozen weights, ensuring that only task specific layers are updated during training.

```
MODEL_NAME = "facebook/wav2vec2-base"
device = torch.device("cuda")

print("Loading feature extractor...")
feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(MODEL_NAME)

print("Loading model...")
model = Wav2Vec2ForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2,
    ignore_mismatched_sizes=True
)

# Set proper label names
model.config.id2label = {0: "bonafide", 1: "spoof"}
model.config.label2id = {"bonafide": 0, "spoof": 1}

# Freeze the CNN feature extractor
for param in model.wav2vec2.feature_extractor.parameters():
    param.requires_grad = False

# Freeze the bottom 6 transformer layers (half of 12)
for i in range(6):
    for param in model.wav2vec2.encoder.layers[i].parameters():
        param.requires_grad = False

# Move model to GPU then convert weights to bfloat16
model = model.to(device)
model = model.to(torch.bfloat16)

# Allow TF32
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True

print(f"Model loaded on {device}")

trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"Trainable parameters: {trainable:,} / {total:,}")
print(f"Frozen parameters: {total - trainable:,} / {total:,}")
print(f"\nLabel mapping: {model.config.id2label}")
```

Figure 39 Model Loading and Layer Freezing

6.2.5 Data Loading and Class Imbalance Handling

This stage defines how training and validation data are structured and fed into the model during optimisation. The dataset objects created in this step wrap the previously defined audio processing pipeline and determine whether augmentation is applied depending on the training phase. Augmentation is enabled for the training dataset to improve generalisation, while it is disabled for the validation set to ensure that evaluation is performed on clean and consistent data.

A significant challenge in this dataset is class imbalance, where the number of Bonafide and spoof samples is not naturally equal. If left unchecked, this imbalance can bias the model towards predicting the majority class more frequently, leading to misleading accuracy scores and poor production performance. To mitigate this, a weighted sampling strategy was used.

Class weights are computed based on the frequency of each class in the training dataset. These weights are then used to construct a PyTorch `WeightedRandomSampler`, which adjusts the probability of each sample being selected during training. This ensures that both classes are presented to the model more evenly, even if their raw distribution is uneven (Hughes, 2022).

The sampler is integrated into the training `DataLoader`, which controls how data is batched and fed into the model. A fixed batch size is used to maintain stable GPU utilisation, while shuffling is replaced by weighted sampling to enforce class balance. For the validation `DataLoader`, sampling has no augmentation applied, ensuring that evaluation metrics reflect true model performance without any artificial variation.

Finally, the system outputs the resulting dataset sizes and computed class weights. This provides transparency into the imbalance correction and confirms that the training pipeline is correctly configured before model optimisation begins. In this case the values are Class weights: bonafide=0.0003875968992248062, spoof=4.3859649122807014e-05. That is a weighting of 8.8x more for bonafide which aligns with the difference in class balance, thus the balancing worked properly.

```

# Cell 7: Create data loaders
from torch.utils.data import WeightedRandomSampler

# Create dataset objects with augmentation
train_dataset = ASVspooDataset(train_df, feature_extractor, augment=True)
dev_dataset = ASVspooDataset(dev_df, feature_extractor, augment=False) # No augmentation for validation

# Handle class imbalance using a weighted sampler
class_counts = train_df["label"].value_counts().sort_index().values
class_weights = 1.0 / class_counts
sample_weights = train_df["label"].map({0: class_weights[0], 1: class_weights[1]}).values

sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(train_dataset),
    replacement=True # Changed to True for better class balance
)

# DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    sampler=sampler,
    num_workers=0,
    pin_memory=True
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=8,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print(f"Training batches: {len(train_loader)}")
print(f"Development batches: {len(dev_loader)}")
print("Improved augmentation enabled for training set")
print(f"Class weights: bonafide={class_weights[0]}, spoof={class_weights[1]}")
✓ 0.0s

```

Figure 40 Weighted Sampling

```

Training batches: 3173
Development batches: 3106
Improved augmentation enabled for training set
Class weights: bonafide=0.0003875968992248062, spoof=4.3859649122807014e-05

```

Figure 41 Class Weight Values

6.2.6 Optimisation Strategy and Evaluation Metrics

This stage defines how the model is trained and evaluated during the learning process. The optimisation strategy uses the Adaptive Moment Estimation with Weight Decay (AdamW) optimiser, which is widely used for transformer architectures due to its ability to handle weight decay more effectively than standard Adam. Only trainable parameters are passed to the optimiser, ensuring that frozen layers remain unchanged during training (Pykes, 2024; Sitanand, 2025).

A learning rate of $1e-4$ is used as a balanced starting point for fine-tuning. This value is small enough to avoid destabilising the pretrained weights, but still allowing meaningful adaptation. Weight decay is also applied to improve generalisation and reduce overfitting.

To further stabilise training, a linear learning rate scheduler with warmup is used. During the initial phase of training, the learning rate gradually increases from a lower value to the target rate. This prevents sudden large updates to model weights, which can negatively

impact convergence. After the warmup period, the learning rate remains stable for the remainder of training.

An evaluation function is also used to measure model performance on the validation dataset. This function runs the model in evaluation mode. For each batch, the model outputs prediction logits (Illuri Sandeep, 2025), which are converted into probabilities using the softmax function (GeeksforGeeks, 2024).

Three key metrics are calculated. Loss provides an indication of how well the model fits the data, while accuracy measures the proportion of correct predictions. The ROC-AUC score is used as a main evaluation metric (Google Developers, 2019). This is particularly important for binary classification problems such as deepfake detection, as it evaluates the model's ability to distinguish between classes across different decision thresholds rather than relying on a fixed cutoff.

```
# AdamW optimiser
optimiser = AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-4,
    weight_decay=0.01
)

# Linear warmup scheduler
total_steps = len(train_loader) * 5 # 5 epochs
warmup_steps = int(0.1 * total_steps)
scheduler = LinearLR(
    optimiser,
    start_factor=0.1,
    end_factor=1.0,
    total_iters=warmup_steps
)

def evaluate(model, loader, device):
    # Run model on dev set and return loss, accuracy and ROC-AUC
    model.eval()
    total_loss, correct, all_labels, all_probs = 0, 0, [], []

    with torch.no_grad():
        for batch in loader:
            input_values = batch["input_values"].to(device)
            labels = batch["label"].to(device)

            outputs = model(input_values=input_values, labels=labels)
            total_loss += outputs.loss.item()

            probs = torch.softmax(outputs.logits, dim=-1).to(torch.float32)
            preds = probs.argmax(dim=-1)
            correct += (preds == labels).sum().item()

            all_labels.extend(labels.cpu().numpy())
            all_probs.extend(probs[:, 1].cpu().numpy())

    avg_loss = total_loss / len(loader)
    accuracy = correct / len(loader.dataset)
    roc_auc = roc_auc_score(all_labels, all_probs)
    return avg_loss, accuracy, roc_auc
```

Figure 42 Optimiser

6.2.7 Training Loop and Model Optimisation

This cell implements the full training loop, which is responsible for iteratively updating the model weights, monitoring performance, and saving the best performing model. It brings together all the previous components, including the dataset, data loaders, optimiser, and evaluation function.

The training process is configured to run for a fixed number of epochs, in this case five. Periodic evaluation is performed every set number of steps. Rather than evaluating only at the end of each epoch, intermediate evaluation is used to provide more consistent insight into model performance and to allow earlier detection of overfitting or stagnation. This is particularly useful when training deep learning models on large datasets, where waiting until the end of an epoch might delay important feedback.

During each training iteration, batches of audio data are passed through the model using mixed precision via automatic casting. This allows computations to be performed using reduced precision, which improves performance on GPUs while keeping numerical stability. The model outputs a loss value, which is then backpropagated to compute gradients. Gradient clipping is applied before updating the model weights, which prevents excessively large updates that could destabilise training.

The optimiser updates only the trainable parameters, while the learning rate scheduler adjusts the learning rate during the warmup phase. After each update step, gradients are reset to ensure that each batch is processed independently.

A key feature of this training loop is the integration of regular evaluation using the development dataset. At fixed intervals, the model is switched to evaluation mode and assessed using the previously defined evaluation function. This produces three metrics: loss, accuracy, and ROC-AUC. ROC-AUC is treated as the primary metric, as it provides a more reliable measure of performance for binary classification problems.

Model checkpointing is based on improvements in ROC-AUC. Whenever a new best score is achieved, the model and feature extractor are saved to disk. If performance does not improve over a number of consecutive evaluations, a patience counter is incremented. Once this counter reaches a threshold, early stopping is triggered, terminating training to prevent overfitting and unnecessary computation.

Another important addition in this implementation is the tracking of augmentation statistics. At the start of each epoch, counters are reset, and during training, each applied augmentation is recorded. At the end of the epoch, a summary is printed showing how frequently each augmentation type was used. This provides visibility into the augmentation

process and helps verify that the intended transformations are being applied at the expected rates.

The training loop also includes periodic logging of training loss, which helps track convergence over time. This allows trends such as steady improvement, plateauing, or instability to be viewed during training.

```
from torch.amp import autocast
from sklearn.metrics import roc_auc_score

EPOCHS = 5
EVAL_STEPS = 200
SAVE_DIR = r"C:\deepfake-project\models\wav2vec2_finetuned_v2"
best_roc_auc = 0.0
patience = 0
PATIENCE_MAX = 4

os.makedirs(SAVE_DIR, exist_ok=True)

print("\n" + "="*60)
print("TRAINING")
print("="*60)
print(f"Save directory: {SAVE_DIR}")
print(f"Eval frequency: every {EVAL_STEPS} steps")
print(f"Early stopping: {PATIENCE_MAX} evaluations without improvement")
print("="*60 + "\n")

for epoch in range(EPOCHS):

    model.train()
    epoch_loss = 0
    step = 0

    # reset augmentation stats each epoch
    AUG_STATS.clear()
    AUG_TOTAL = 0

    print(f"\n===== EPOCH {epoch+1} START =====\n")

    for batch in train_loader:

        input_values = batch["input_values"].to(device)
        labels = batch["label"].to(device)

        with autocast(device_type="cuda", dtype=torch.bfloat16):

            outputs = model(input_values=input_values, labels=labels)

        loss = outputs.loss

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimiser.step()
        scheduler.step()
        optimiser.zero_grad()

        epoch_loss += loss.item()
        step += 1

    if step % 50 == 0:
        avg_loss = epoch_loss / step
        print(f"Epoch {epoch+1} | Step {step}/{len(train_loader)} | Loss: {avg_loss:.4f}")

    # Evaluation
    if step % EVAL_STEPS == 0:

        dev_loss, dev_acc, dev_roc = evaluate(model, dev_loader, device)

        print(f"\n>>> Eval @ step {step} | Loss: {dev_loss:.4f} | Acc: {dev_acc:.4f} | ROC-AUC: {dev_roc:.4f}")

        if dev_roc > best_roc_auc:
            best_roc_auc = dev_roc
            patience = 0
            model.save_pretrained(SAVE_DIR)
            feature_extractor.save_pretrained(SAVE_DIR)
            print(f"    New best model saved (ROC-AUC: {best_roc_auc:.4f})")
        else:
            patience += 1
            print(f"    No improvement - patience {patience}/{PATIENCE_MAX}")
            if patience >= PATIENCE_MAX:
                print("\nEarly stopping triggered")
                break

        model.train()

    # Augmentation Stats Report
    print("\n===== AUGMENTATION STATS =====")
    print(f"Total augmented samples: {AUG_TOTAL}")

    if AUG_TOTAL > 0:
        for k, v in AUG_STATS.items():
```

Figure 43 Full Training Loop

6.3 LSTM Model Training and Development

A second deep learning architecture was implemented using a bidirectional Long Short-Term Memory (LSTM) network (Stryker, n.d.; GeeksforGeeks, 2024). This model was designed to evaluate how a more traditional sequential architecture performs on the same synthetic speech detection task. Unlike Wav2Vec2, which learns directly from raw waveforms, the LSTM model works on Mel-spectrogram representations of audio, making it dependent on explicit feature extraction prior to training.

This section focuses only on the differences in implementation, particularly the use of spectrogram inputs and the LSTM architecture.

6.3.1 Mel-Spectrogram Feature Representation

Unlike the Wav2Vec2 pipeline, which consumes raw waveform input, the LSTM model requires a structured time–frequency representation of audio. A Mel-spectrogram transformation was introduced using torchaudio (Roberts, 2020).

A Mel-spectrogram converts the raw waveform into a two-dimensional representation where the x-axis represents time and the y-axis represents frequency bands. This transformation is critical because LSTMs are designed to process sequential features rather than raw audio signals.

A relatively small Fast Fourier Transform (FFT) window size is used to preserve short-term speech characteristics, while the hop length determines the overlap between frames, ensuring smooth transitions.

This transformation effectively converts each audio sample into a sequence of feature vectors, making it suitable for recurrent neural network processing.

```
import torchaudio

SAMPLE_RATE = 16000

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=SAMPLE_RATE,
    n_mels=80,
    n_fft=1024,
    hop_length=320
)
```

Figure 44 Mel-Spectrogram Transformation

6.3.2 Dataset Adaptation for Spectrogram

While the overall dataset structure remains identical to the Wav2Vec pipeline, the LSTM implementation introduces a key difference in how audio is processed before being passed to the model.

Instead of returning raw waveform or feature extractor outputs, each audio file is transformed into a Mel-spectrogram during the dataset loading stage.

After loading augmentation, the audio signal is:

1. Truncated or padded to a fixed length of four seconds.
2. Normalised to prevent amplitude bias.
3. Converted into a Mel-spectrogram representation.
4. Log-scaled to compress dynamic range and improve stability.

The spectrogram is then transposed so that time becomes the primary sequence dimension, allowing it to be processed by the LSTM.

To ensure consistent input sizes across all samples, fixed-length padding or truncation is applied along the time dimension. This is particularly important because recurrent networks require uniform sequence lengths within a batch.

Unlike the Wav2Vec pipeline, no external feature extractor is used, as feature extraction is fully handled within this preprocessing stage.

```
if len(audio) >= MAX_SAMPLES:
    audio = audio[:MAX_SAMPLES]
else:
    audio = np.pad(audio, (0, MAX_SAMPLES - len(audio)))

peak = np.abs(audio).max()
if peak > 0:
    audio = audio / peak

audio = torch.from_numpy(audio).float()

mel = mel_transform(audio)
mel = torch.log(mel + 1e-6)

mel = mel.transpose(0, 1) # (time, n_mels)

max_len = MAX_SAMPLES // 320

if mel.shape[0] > max_len:
    mel = mel[:max_len]
else:
    pad_len = max_len - mel.shape[0]
    mel = torch.nn.functional.pad(mel, (0, 0, 0, pad_len))

return {
    "input_values": mel.to(torch.bfloat16),
    "label": torch.tensor(row["label"], dtype=torch.long)
}
```

Figure 45 Mel-Spectrogram on the Dataset

6.3.3 LSTM Model Architecture Design

The LSTM model was designed as a light sequential classifier capable of learning speech patterns. The architecture consists of two main components: an LSTM encoder followed by a classification head.

The LSTM processes input sequences in both forward and backward directions, allowing the model to capture contextual dependencies from both past and future time steps.

A multi-layer configuration was used to increase capacity, while dropout is applied between layers to reduce overfitting. The hidden dimensionality is set to a moderate size to balance performance and efficiency.

The output of the LSTM is aggregated using mean pooling across the time. This converts the variable-length sequence output into a fixed-size representation, which is then passed into a classifier.

The classifier consists of a two-layer network with a Rectified Linear Unit (ReLU) activation function and dropout regularisation. The final output layer produces logits for binary classification.

Unlike the Wav2Vec model, this architecture does not use pretrained representations. As a result, it learns directly from spectrogram patterns, making it more sensitive to feature quality but less expensive to run.

```
import torch

device = torch.device("cuda")

class LSTMClassifier(torch.nn.Module):
    def __init__(self, input_dim=80, hidden_dim=256):
        super().__init__()

        self.lstm = torch.nn.LSTM(
            input_dim,
            hidden_dim,
            num_layers=2,
            batch_first=True,
            bidirectional=True,
            dropout=0.3
        )

        self.classifier = torch.nn.Sequential(
            torch.nn.Linear(hidden_dim * 2, 128),
            torch.nn.ReLU(),
            torch.nn.Dropout(0.3),
            torch.nn.Linear(128, 2)
        )

    def forward(self, x, labels=None):
        out, _ = self.lstm(x)
        out = out.mean(dim=1)

        logits = self.classifier(out)

        loss = None
        if labels is not None:
            loss = torch.nn.functional.cross_entropy(logits, labels)

        return {"loss": loss, "logits": logits}

# Instantiate model
model = LSTMClassifier().to(device).to(torch.bfloat16)
```

Figure 46 LSTM Model Code

6.3.4 Training Configuration and Data Loading Differences

The data loading process remains similar to the previous model, including the use of weighted sampling to address class imbalance. However, a key difference is that no feature extractor is required, as spectrograms are already generated within the dataset pipeline.

Unlike the Wav2Vec setup, the model input is already in tensor form, meaning batching is more straightforward.

```
from torch.utils.data import WeightedRandomSampler, DataLoader

# Create dataset objects with augmentation
train_dataset = ASvspoofDataset(train_df, feature_extractor=None, augment=True)
dev_dataset = ASvspoofDataset(dev_df, feature_extractor=None, augment=False)

# Handle class imbalance using weighted sampler
class_counts = train_df["label"].value_counts().sort_index().values
class_weights = 1.0 / class_counts
sample_weights = train_df["label"].map({0: class_weights[0], 1: class_weights[1]}).values

sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(train_dataset),
    replacement=True
)

# Dataloaders
train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    sampler=sampler,
    num_workers=0,
    pin_memory=True
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=8,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print(f"Training batches: {len(train_loader)}")
print(f"Development batches: {len(dev_loader)}")
print("LSTM pipeline active (no feature_extractor required)")
print(f"Class weights: bonafide={class_weights[0]:.4f}, spoof={class_weights[1]:.4f}")
```

Figure 47 Weighted Sampler for LSTM

6.3.5 Training Loop and Evaluation Differences

The training loop for the LSTM model closely follows the structure used in the Wav2Vec implementation. However, there are several key differences in implementation.

First, evaluation is performed manually within the training loop rather than through a separate evaluation function. This was done to simplify debugging.

During evaluation, probabilities are explicitly converted from bfloat16 to float32. This prevents numerical instability when calculating ROC-AUC and ensures compatibility with scikit-learn evaluation functions.

The model is saved using state_dict rather than full model serialization. Early stopping is again implemented using ROC-AUC as the primary metric. If performance does not improve over a fixed number of evaluation cycles, training is halted to prevent overfitting.

```

# Evaluation
if step % EVAL_STEPS == 0:

    model.eval()

    total_loss = 0
    correct = 0
    all_labels = []
    all_probs = []

    with torch.no_grad():
        for batch in dev_loader:

            input_values = batch["input_values"].to(device)
            labels = batch["label"].to(device)

            outputs = model(input_values, labels)

            logits = outputs["logits"]
            loss = outputs["loss"]

            total_loss += loss.item()

            # convert bfloat16 to float32 before numpy
            probs = torch.softmax(logits, dim=-1).float()
            preds = probs.argmax(dim=-1)

            correct += (preds == labels).sum().item()

            all_labels.extend(labels.detach().cpu().numpy())
            all_probs.extend(probs[:, 1].detach().cpu().numpy())

    dev_loss = total_loss / len(dev_loader)
    dev_acc = correct / len(dev_loader.dataset)
    dev_roc = roc_auc_score(all_labels, all_probs)

    print(f"\n>>> Eval @ step {step}")
    print(f"Loss: {dev_loss:.4f} | Acc: {dev_acc:.4f} | ROC-AUC: {dev_roc:.4f}")

```

Figure 48 Updated Evaluation for LSTM training

6.4 PANNs CNN14 Model Selection and Setup

The Pretrained Audio Neural Networks (PANNs) CNN14 architecture was selected as a third model to provide a strong baseline for deepfake detection using spectrogram representations. CNN14 is designed to learn time–frequency patterns from Mel-spectrogram inputs. This makes it particularly well-suited for audio classification tasks (Olamendy, 2023).

6.4.1 Input Representation Difference

A key architectural difference in this pipeline is the use of Mel-spectrogram features instead of raw audio. The audio is transformed using a `torchaudio.transforms.MelSpectrogram` layer with 80 Mel bins, a 1024 FFT window, and a hop length of 320 samples. This produces a compact time–frequency representation that is more aligned with CNN feature extraction.

Log scaling is applied to the spectrogram ($\log(\text{mel} + 1\text{e-}6)$) to compress dynamic range and improve stability. The spectrogram is transposed into a time-major format before being

padded or truncated to a fixed length. This ensures consistent tensor shapes across batches.

6.4.2 Dataset Design Differences

The dataset implementation is simplified compared to earlier models because it does not require Hugging Face feature extraction or sequence modelling preparation. Each audio file is directly converted into a waveform tensor and then transformed into a Mel-spectrogram within the dataset class itself.

A key distinction is that the returned tensor is already in a CNN-ready format ([time, mel_bins]), meaning no additional preprocessing is required during training.

Augmentation is still applied, but it operates on raw waveform before spectrogram conversion.

```
def fix_length(self, audio):
    if len(audio) > MAX_SAMPLES:
        audio = audio[:MAX_SAMPLES]
    else:
        audio = np.pad(audio, (0, MAX_SAMPLES - len(audio)))
    return audio

def normalize(self, audio):
    peak = np.abs(audio).max()
    return audio / peak if peak > 0 else audio

def __getitem__(self, idx):
    row = self.df.iloc[idx]
    audio = self.load_audio(row["path"])

    if self.augment and not row["path"].endswith(".ogg"):
        audio = self.augment_audio(audio)

    audio = self.fix_length(audio)
    audio = self.normalize(audio)

    return {
        "input": torch.tensor(audio, dtype=torch.float32),
        "label": torch.tensor(row["label"], dtype=torch.long)
    }
```

Figure 49 CNN Audio Augment

6.4.3 Model Forward Pass and Output Handling

The forward pass of CNN14 differs significantly from both previous models. Instead of returning a structured object with logits and loss or a custom dictionary output, CNN14 returns a dictionary containing clipwise_output, which represents class probabilities.

Because of this, the loss function is computed manually using cross_entropy (Brownlee, 2019), on the extracted logits. During evaluation, predictions are gotten directly from the softmax of clipwise_output.

```

with autocast(device_type="cuda", dtype=torch.bfloat16):
    outputs = model(inputs)
    logits = outputs["clipwise_output"]
    loss = torch.nn.functional.cross_entropy(logits, labels)

loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimiser.step()
scheduler.step()
optimiser.zero_grad()

epoch_loss += loss.item()
step += 1

```

Figure 50 Cross entropy for loss CNN

7. Evaluation

7.1 Model Evaluation

This section evaluates the performance of the deep learning models developed throughout this project:

- Wav2Vec2
- LSTM
- CNN14 PANNs

Evaluation focuses on training behaviour, convergence, and classification performance using loss, accuracy, and ROC-AUC metrics.

7.1.1 Evaluation Metrics Overview

The following metrics were used:

- Loss: Measures model error during training
- Accuracy: Percentage of correct predictions at a fixed threshold
- ROC-AUC: Primary metric, measuring class separability across all thresholds

ROC-AUC is particularly important for deepfake detection, as it reflects how well the model distinguishes real vs synthetic speech independent of a fixed decision boundary.

7.1.2 Wav2Vec2 Model Performance Analysis

The Wav2Vec2 model achieved the strongest performance overall due to benefiting from pretrained speech representations and direct waveform input.

Key Observations:

- **Fast Convergence:**
The model showed rapid improvement early in training, indicating effective transfer learning from pretrained weights.
- **High ROC-AUC Performance:**
Achieved the highest ROC-AUC among all models demonstrating excellent class separation.
- **Stable Training Behaviour:**
Loss decreased smoothly with minimal fluctuations, indicating stable optimisation.
- **Strong Generalisation:**
Performance remained consistent across validation steps, suggesting robustness to unseen data.

Training Curve Insights (Wav2Vec2)

Loss Curve

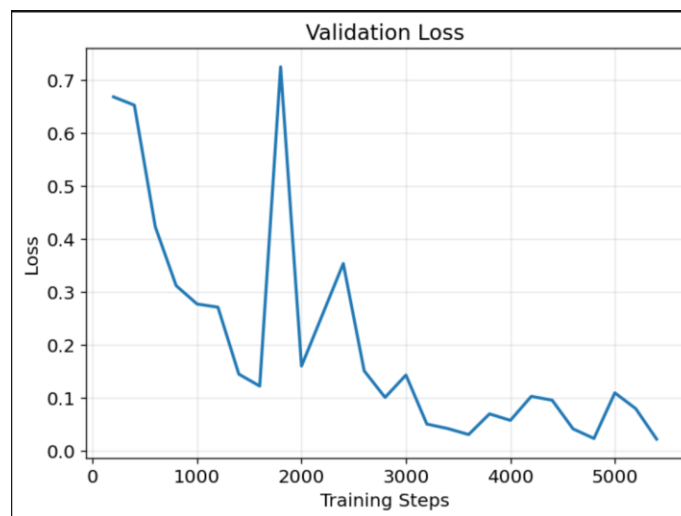


Figure 51 Wav2Vec Loss Curve

- Relatively consistent downward trend with some instability
- Indicates stable gradient updates and effective fine-tuning

Accuracy Curve

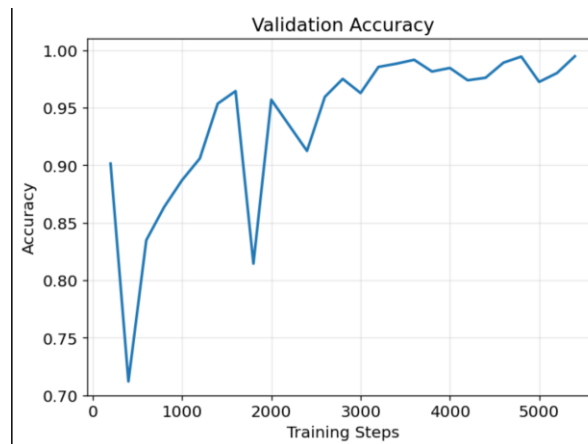


Figure 52 Wav2Vec Accuracy Curve

- Rapid increase in early stages
- High and stable plateau
- Reflects strong classification performance

ROC-AUC Curve

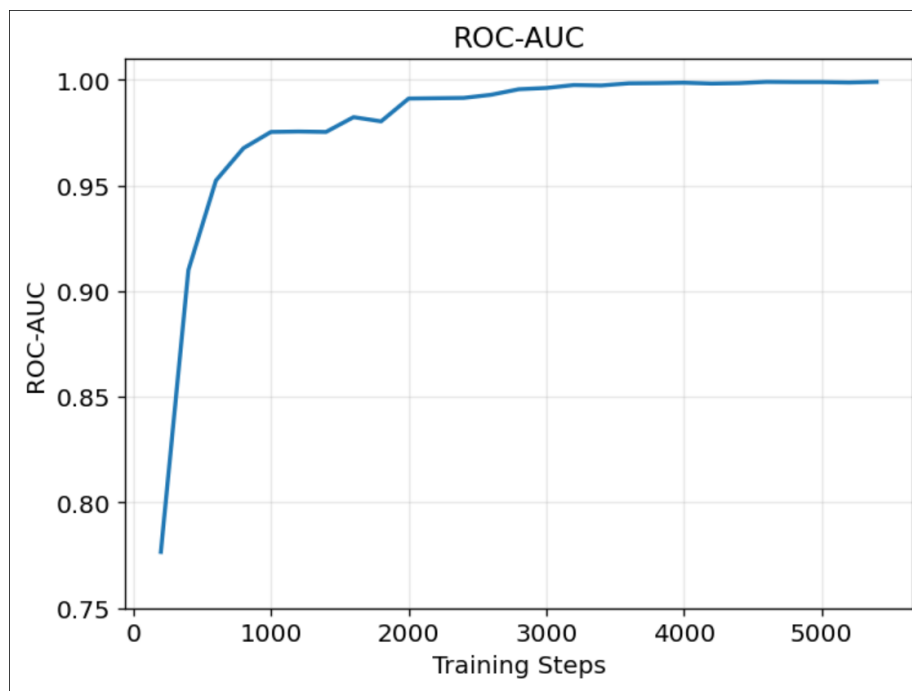


Figure 53 Wav2Vec ROC-AUC Curve

- Rapid improvement early in training

- Reaches high values quickly (>0.95)
- Remains stable with minor improvements

The strong performance of Wav2Vec2 is primarily due to:

- Learning from raw waveform data
- Leveraging pretrained speech representations

This makes it significantly more robust to real-world distortions such as compression and noise.

7.1.3 LSTM Model Performance Analysis

The LSTM model provided strong training performance despite being significantly simpler.

Key Observations:

- **Steady Loss Reduction:**
Loss decreased from ~ 0.69 to ~ 0.39 , showing effective learning.
- **High ROC-AUC Performance:**
Achieved a peak ROC-AUC of 0.9577, indicating strong discriminative ability.
- **Moderate Accuracy:**
Accuracy stabilised around 80–83%, lower than Wav2Vec2.
- **Early Stopping Triggered:**
Training stopped in Epoch 3 due to plateauing performance.

Training Curve Insights (LSTM)

Loss Curve

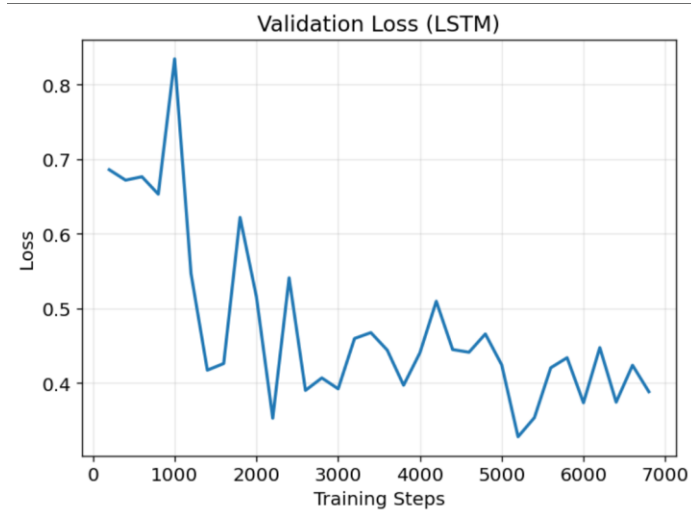


Figure 54 LSTM Loss Curve

- Clear downward trend
- Slight fluctuations in later stages
- Indicates convergence with some instability

Accuracy Curve

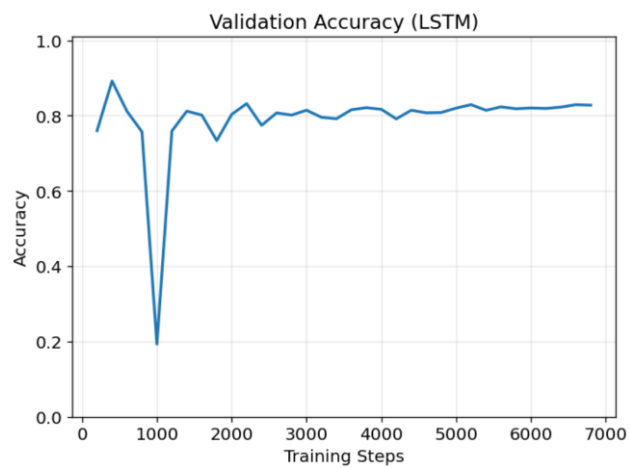


Figure 55 LSTM Accuracy Curve

- Rapid early improvement
- Plateau in later stages
- Reflects saturation of learning capacity

ROC-AUC Curve

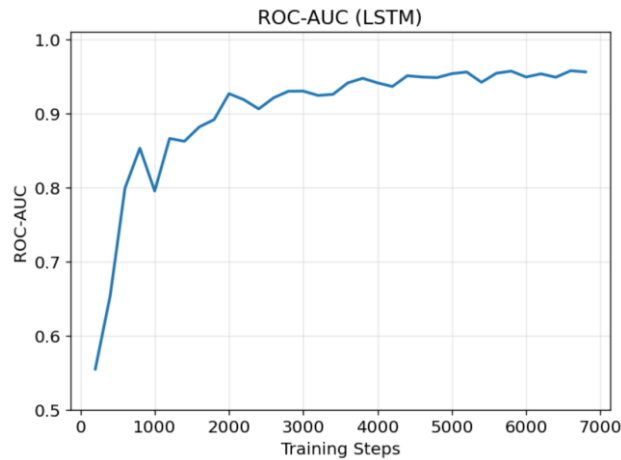


Figure 56 ROC-AUC Curve LSTM

- Strong upward trend
- Peaks around 0.9577
- Diminishing improvements toward end

Key Insight

The LSTM model performs well but:

- Depends heavily on Mel-spectrogram quality
- Less robust to noise compared to Wav2Vec2
- Would require more preprocessing and feature engineering, and likely a more complex model

7.1.4 CNN14 (PANNs) Model Performance Analysis

The CNN14 model provided a strong middle-ground between LSTM and Wav2Vec2, using convolutional feature extraction on spectrogram inputs.

Key Observations:

- **Efficient Feature Learning:**
CNN layers effectively captured time–frequency patterns in spectrograms.
- **Strong ROC-AUC Performance:**
Performance typically excelled over that of LSTM, though slightly below Wav2Vec2.

- **Faster Training:**
Converged faster than LSTM and Wav2Vec.
- **Stable Accuracy:**
Accuracy remained consistent, with fewer fluctuations than LSTM.

Training Curve Insights (CNN14)

Loss Curve

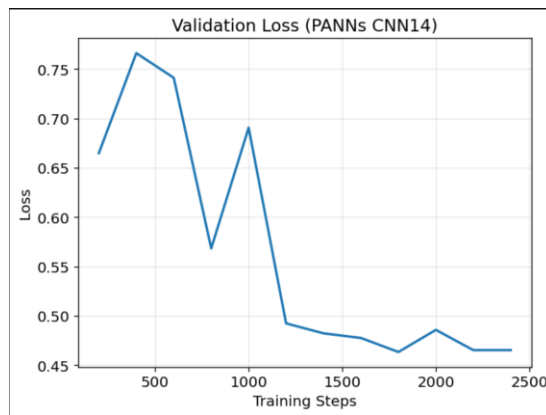


Figure 57 CNN Loss Curve

- Smooth decrease
- Slight plateau toward later stages

Accuracy Curve

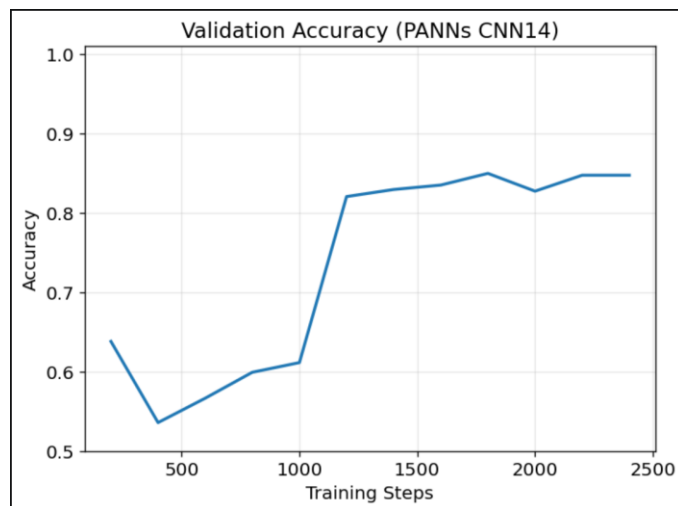


Figure 58 CNN Accuracy Curve

- Steady improvement
- More stable than LSTM
- Lower peak than Wav2Vec2

ROC-AUC Curve

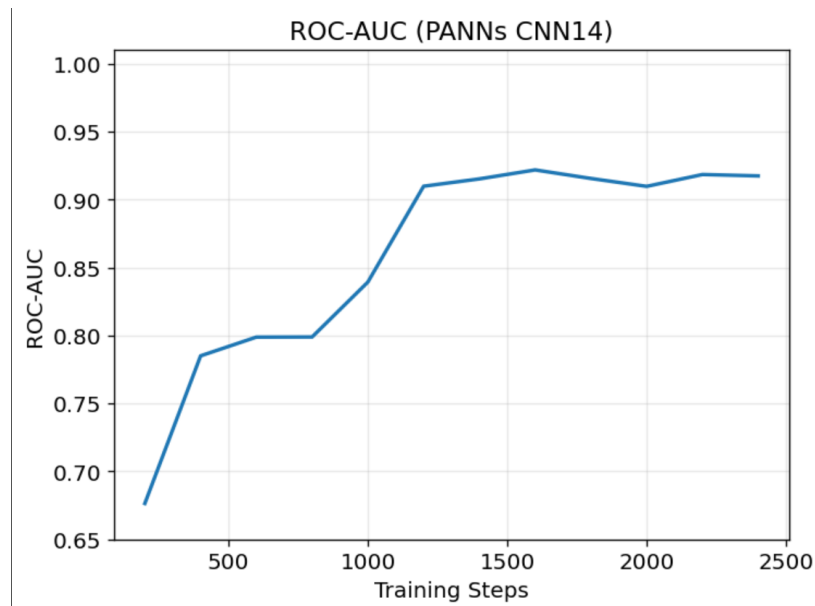


Figure 59 ROC-AUC CNN Curve

- Strong upward trend
- Reaches high values approx. 0.94–0.96 range
- Slightly below Wav2Vec2

Key Insight

CNN14 benefits from:

- Strong local pattern detection in spectrograms
- Efficient training
- Less dependence on sequential modelling

However, it is still limited by:

- Indirect input representation
- Less contextual understanding compared to transformers

7.1.5 Overall Findings

Key Comparative Insights:

- Wav2Vec2 clearly outperforms other models, due to pretrained knowledge and raw audio processing.
- LSTM performs surprisingly well despite its simplicity
- CNN14 offers a balance between performance and efficiency.
- Pretrained transformer models significantly improve performance.
- Spectrogram models remain competitive but less robust.

All three models demonstrated capability in detecting synthetic speech. The LSTM and CNN14 models achieved high ROC-AUC scores, confirming that meaningful patterns exist within spectrogram representations.

However, the Wav2Vec2 model consistently achieved the best performance due to its ability to learn directly from raw waveform data and leverage pretrained speech representations. This makes it the most suitable model for real-world deployment in this system.

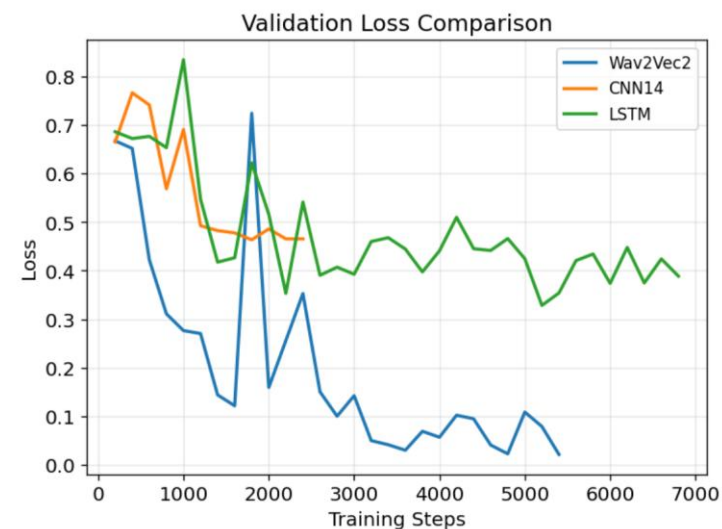


Figure 60 Comparative Loss Curve

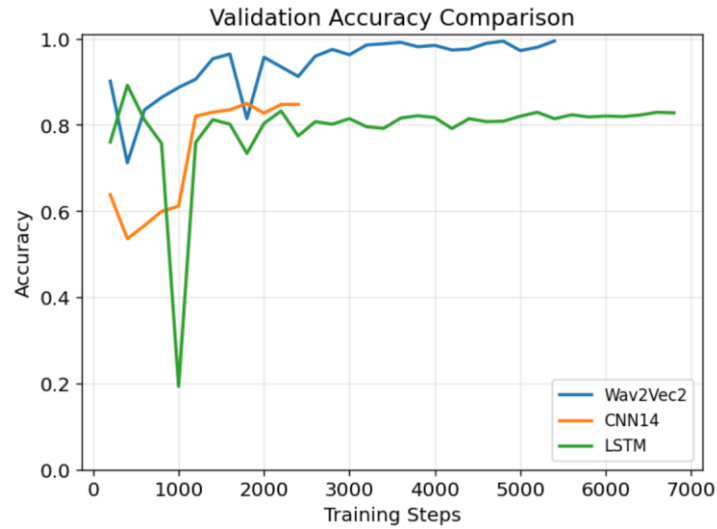


Figure 61 Comparative Accuracy Curve

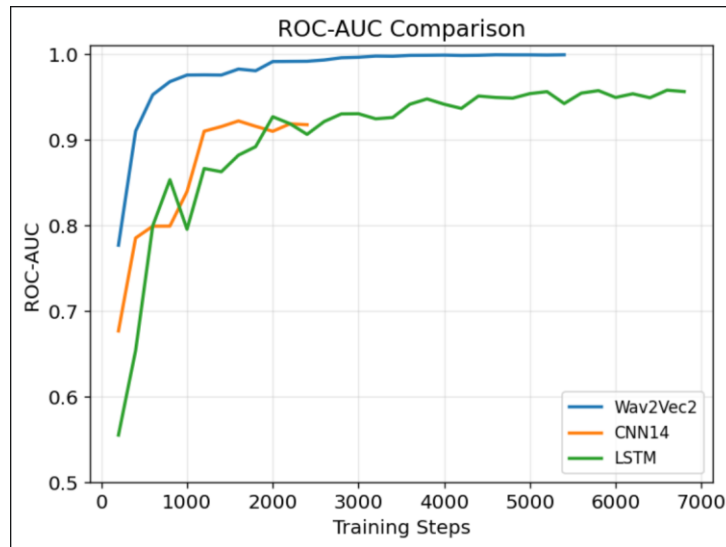


Figure 62 Comparative ROC-AUC Curve

7.2 System Evaluation

This section evaluates the performance of the complete end-to-end system, including the WhatsApp chatbot interface, the web interface, the backend processing pipeline, and the deployed machine learning model. Unlike the model evaluation presented in Section 6.1, this section focuses on real system behaviour and performance under different input conditions, particularly variations in audio quality, compression, and noise.

7.2.1 WhatsApp Integration Performance

The WhatsApp chatbot interface was successfully implemented and remained stable throughout testing. The bot consistently handled incoming voice messages, correctly

triggered the backend pipeline, and returned structured responses containing both the prediction label and confidence scores. From a system perspective, the end-to-end pipeline functions as intended:

- Voice notes are successfully received and downloaded from WhatsApp
- Audio is correctly converted and preprocessed using FFmpeg
- Requests are reliably sent to the FastAPI backend
- Responses are returned and formatted correctly to the user
- The system remains stable under repeated use

However, while the pipeline is operational, there is significant performance degradation is at the model level when processing WhatsApp audio inputs. WhatsApp voice notes are heavily compressed using the Opus codec, which introduces artefacts, bandwidth limitations, and loss of high-frequency detail. This creates a shift between the training data and real audio which includes some augmentation to this issue but ultimately failed to prepare the model successfully.

As a result, the model frequently fails to correctly classify WhatsApp voice messages. In many cases, real human speech is misclassified as synthetic due to distortion.

This indicates that the limitation is not in the system pipeline or bot implementation, but in model generalisation to heavily compressed real-world audio. As can be seen visually below, the much of the audio features are completely lost due to the compression, and even with decompression, the data remains lossy, so lost features are no longer accessible to the model.

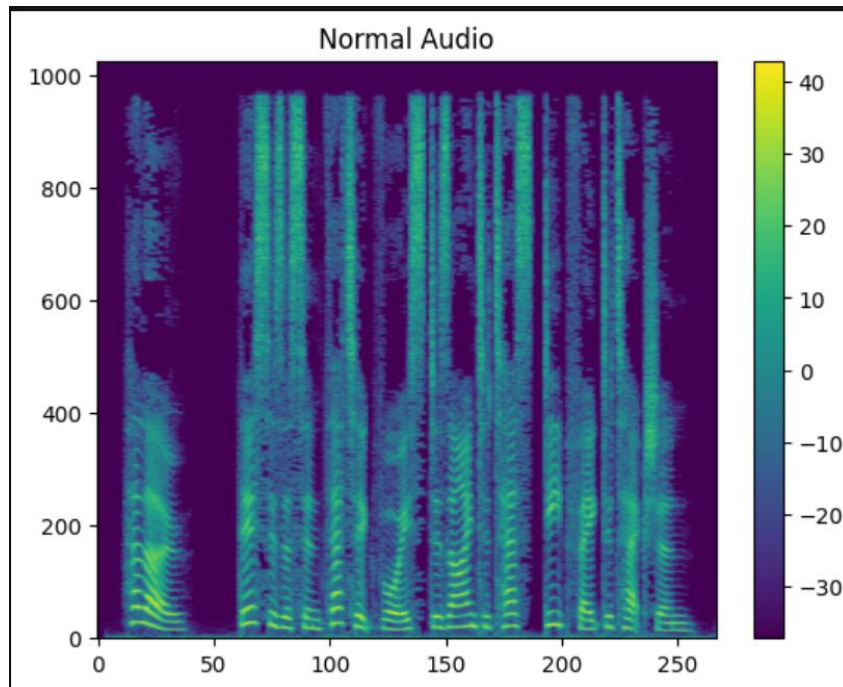


Figure 63 Uncompressed Audio Visualisation

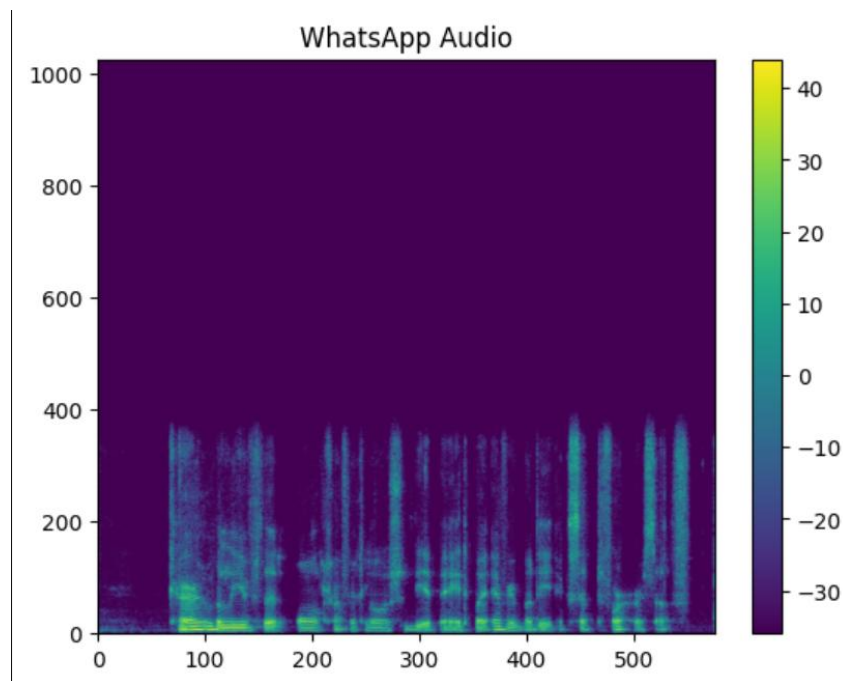


Figure 64 WhatsApp audio Visualisation

7.2.2 Web Interface Performance (High-Quality Audio Testing)

In contrast to the WhatsApp interface, the web upload system provides a controlled environment where audio quality can be significantly higher and more consistent. This allows for a clearer evaluation of model capability without the effects of compression artifacts.

Testing on the web interface showed substantially improved performance:

- Long-form audio inputs e.g. 5-minute podcast clips, were correctly classified
- High-quality human speech was consistently identified as real)
- High-quality AI-generated speech samples were correctly identified as synthetic
- The model demonstrated strong discrimination when audio quality is preserved

These results indicate that the underlying model architecture is effective when provided with clean or studio-quality input data. In particular, the Wav2Vec2 model demonstrates strong robustness to speaker variation and content diversity when signal quality is not degraded.

7.2.3 Impact of Audio Quality and Compression

A key finding from system-level evaluation is that audio quality has a greater impact on performance than content or speaker identity. The system performs well under the following conditions:

- Studio-quality recordings
- Clean microphone input
- Minimal background noise

However, performance degrades significantly under the following conditions:

- WhatsApp Opus compression
- Low-quality mobile microphones
- Background noise or environmental interference
- Heavily compressed voice messages

This indicates that the model has learned strong feature representations for high-quality speech but is sensitive to signal degradation and codec distortion.

7.2.4 Effectiveness of Windowing Strategy

The windowing mechanism implemented in the backend plays an important role in improving system robustness, particularly for longer audio inputs. By segmenting audio into overlapping fixed-length windows, the system is able to do the following:

- Isolate higher-quality segments within noisy audio
- Reduce the impact of small distortions
- Aggregate predictions across multiple parts of the audio signal

This approach is particularly effective for longer audio clips such as podcast recordings or extended speech, where certain segments remain clean even if others are degraded. In several cases, longer inputs produced more accurate predictions than shorter voice samples, as the model was able to leverage multiple windows and average out noisy predictions.

This suggests that the windowing strategy partially compensates for real-world audio variability and is a valuable component of the system design.

7.2.5 System-Level Summary

Overall, the system successfully achieves its core objective of providing an end-to-end deepfake voice detection pipeline integrated into real-world communication platforms. The implementation demonstrates:

- A fully functional WhatsApp chatbot interface
- A stable and scalable FastAPI backend
- A working cloud inference architecture
- Integration of machine learning models

However, evaluation also highlights a clear limitation in model generalisation. While the system performs well on clean, high-quality audio, it struggles significantly with compressed and noisy real-world audio.

Addressing this gap would likely require:

- Increased training on real WhatsApp compressed audio
- More aggressive codec simulation during augmentation

- Adaptation techniques for low-quality speech
- Potential fine-tuning on in-the-wild voice data

8. Conclusion

This project successfully designed and implemented an end-to-end deepfake voice detection system integrated into a communication platforms. The system combines a WhatsApp chatbot interface, a FastAPI backend hosted on AWS EC2, and a deep learning model pipeline deployed via AWS EC2. In addition, a secondary web interface was developed to support testing and evaluation.

The primary objective of the project was to investigate whether deep learning models could be effectively integrated into a widely used messaging platform to detect synthetic speech in an accessible and user-friendly manner. This objective was achieved through the development of a complete pipeline that allows users to submit voice messages and receive real-time predictions with a confidence score indicating whether the audio is likely to be real or synthetically generated.

Three machine learning models were implemented and evaluated: a fine-tuned Wav2Vec2 transformer, a bidirectional LSTM using Mel-spectrogram features, and a CNN14 (PANNs) architecture. Across all experiments, Wav2Vec2 consistently demonstrated the strongest. The LSTM and CNN14 models also achieved competitive results, confirming that meaningful patterns exist in spectrogram representations of speech.

However, system-level evaluation revealed a key limitation. While the models perform well on clean, high-quality audio such as podcast recordings or studio generated speech, performance degrades significantly when exposed to compressed or noisy real-world audio, particularly WhatsApp voice messages. This is mainly due to codec compression artifacts and environmental noise as well as audio quality. Despite this, the system architecture itself remains stable and fully functional, with reliable communication between all components and consistent end-to-end processing.

Overall, the project demonstrates the possibility of integrating deep learning audio classification into widely used messaging platforms. It highlights both the strengths of current speech models and the challenges associated with real-world deployment where no audio quality control is possible. The system provides a strong foundation for further research into practical deepfake voice detection tools that are accessible to everyday users.

9. Future Work

While the current system provides a fully functional prototype of a deepfake voice detection platform, several areas for improvement and extension have been identified from the work done.

A primary limitation of the current system is its sensitivity to audio compression and background noise. Future work should focus on improving robustness to real-world audio by expanding the training dataset to include a larger proportion of natural voice recordings, particularly those generated through messaging platforms such as WhatsApp. This would help reduce the gap between training and deployment data. This may require collaboration with WhatsApp to get the amount of data needed, and consent from users who could opt-in to the process.

More advanced data augmentation strategies could be introduced. Although the current system includes compression simulation, noise injection, and pitch variation, future improvements could incorporate more realistic augmentation, including variable bitrate encoding, packet loss simulation, and specific device microphone distortion effects.

Techniques such as adversarial training or unsupervised domain adaptation could be explored to improve generalisation to unseen audio. Also, a multi-stage model could be introduced, where a preprocessing classifier first deduces audio quality before choosing an appropriate detection strategy.

From a system perspective, the current WhatsApp integration relies on unofficial WhatsApp Web libraries, which may have long-term stability issues. A production version of this system would benefit from integration with the official WhatsApp Business API, which requires a business account.

Enhancements could also include real-time call analysis, allowing live calls to be evaluated as they are happening. This would significantly improve the use case in practical scenarios.

Finally, explainability techniques could be integrated into the model. Providing users with deeper feedback, such as highlighting suspicious audio segments, would improve transparency and trust in the system's predictions.

Overall, future work should focus on improving robustness, scalability, and interpretability, making the system closer to a deployable security tool capable of operating reliably with diverse audio.

Glossary

Term / Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASVspoof	Automatic Speaker Verification Spoofing Challenge dataset used for synthetic speech detection research
Augmentation	Artificial modification of training data to improve model robustness and generalisation
AWS	Amazon Web Services
Backend	Server-side component responsible for processing requests
Bandpass Filter	Audio filter that keeps frequencies within a specific range while removing others
Batch Size	Number of samples processed at the same time during model training
bfloat16	A reduced precision floating-point format used to improve deep learning performance and memory efficiency
Binary Classification	A machine learning task involving two output classes, such as real or fake speech
CNN	Convolutional Neural Network
CNN14	A convolutional neural network architecture from the PANNs audio classification framework
Codec	Software or algorithm used to compress/decompress audio
Compression Artefacts	Distortions introduced into audio during compression and encoding
Convergence	The process where model training stabilises

Term / Acronym	Meaning
Cross-Entropy Loss	A loss function used in classification tasks to measure prediction error
CUDA	Compute Unified Device Architecture, NVIDIA's platform for GPU acceleration
DataLoader	PyTorch utility used for batching and loading training data
Deepfake Audio	Artificially generated or manipulated speech designed to imitate real voices
Domain Adaptation	Techniques for improving model performance on different data distributions
EC2	Elastic Compute Cloud, Amazon's virtual server hosting platform
Epoch	One complete pass through the entire training dataset
FastAPI	A Python web framework for building APIs and backend services
Feature Extraction	The process of converting raw data into useful representations for machine learning
FFT	Fast Fourier Transform, an algorithm used to analyse frequency in signals
FFmpeg	Open-source framework used for audio and video processing
Fine-Tuning	Adapting a pretrained model to a specific task using additional training
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
Inference	The process of using a trained machine learning model to make predictions

Term / Acronym	Meaning
Inference Pipeline	The sequence of steps used to generate predictions from new input
JSON	JavaScript Object Notation, a lightweight format for structured data exchange
Learning Rate	A parameter controlling how much model weights are updated during training
Logits	Raw model output values before conversion into probabilities
Loss	Difference between the models output and the actual truth, lower is better
LSTM	Long Short-Term Memory network, a type of recurrent neural network
Mel-Spectrogram	A time-frequency representation of audio based on the Mel frequency scale
MFCC	Mel-Frequency Cepstral Coefficients, handcrafted audio features
Mixed Precision Training	Training using reduced numerical precision to improve performance and reduce memory usage
Model Generalisation	The ability of a model to perform well on unseen data
Ngrok	A tunnelling service used to expose local servers to the internet
OGG	Compressed audio file format commonly used by WhatsApp voice notes
Opus Codec	Audio compression format optimised for speech transmission
Padding	Adding extra values to standardise input length
PANNs	Pretrained Audio Neural Networks
PCM	Pulse-Code Modulation, a standard digital audio representation

Term / Acronym	Meaning
Pitch Shifting	Altering the pitch of audio without changing duration
Plateauing	When model performance stops significantly improving
Pretrained Model	A model previously trained on large datasets before specific fine-tuning
PyTorch	Open-source deep learning framework
Raw Waveform	Original audio signal before feature extraction
ReLU	Rectified Linear Unit activation function
Resampling	Changing the sampling rate of audio
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve, a performance metric for binary classification “probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative” (Google Developers, 2019). Higher is better
SageMaker	Amazon Web Services platform for deploying and hosting machine learning models
Sampling Rate	Number of audio samples captured per second
Sequential Modelling	Learning dependencies in ordered data such as speech
Signal Degradation	Loss of audio quality due to compression or noise
Softmax Function	Function that converts logits into probability values
Spectrogram	Visual representation of audio frequencies over time
Tensor	Multi-dimensional numerical array used in deep learning
Transformer	Deep learning architecture
Validation Set	Dataset used to evaluate model performance during training
Vishing	Voice phishing using phone calls or voice messages

Term / Acronym	Meaning
WAV	Uncompressed audio file format
Wav2Vec2	Transformer speech representation model developed by Meta AI
WeightedRandomSampler	PyTorch sampling method used to address class imbalance
Windowing	Splitting long audio into smaller overlapping segments for processing
Adversarial Training	Training using difficult or manipulated samples to improve robustness

Bibliography

Yamagishi, J., Wang, X., Todisco, M., Sahidullah, M., Patino, J., Nautsch, A., Liu, X., Lee, K.A., Kinnunen, T., Evans, N. and Delgado, H. (2021). *ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection*. [online] arXiv.org. Available at:

<https://arxiv.org/abs/2109.00537>

wwwebjs.dev. (n.d.). *whatsapp-web.js*. [online] Available at: <https://wwwebjs.dev/>.

Brownlee, J. (2019). *A Gentle Introduction to Cross-Entropy for Machine Learning*. [online] Machine Learning Mastery. Available at: <https://machinelearningmastery.com/cross-entropy-for-machine-learning/>

Illuri Sandeep (2025). *Logits vs. Probabilities: Understanding Neural Network Outputs Clearly*. [online] Medium. Available at: <https://illuri-sandeep5454.medium.com/logits-vs-probabilities-understanding-neural-network-outputs-clearly-0e86a4256a0e>

Tech With Tim (2025) *How To Build an API with Python (LLM Integration, FastAPI, Ollama & More)* [YouTube video]. Available at: <https://www.youtube.com/watch?v=cy6EAp4iNN4> (Accessed: 2 December 2025)

Meta (n.d.) *WhatsApp Cloud API Documentation*. Available at: <https://developers.facebook.com/docs/whatsapp/cloud-api> (Accessed: 2 December 2025)

Solanki, R. et al. (n.d.) *Artificial Intelligence to Combat Audio Fraud: A Flask-Deployed Hybrid Deep Learning System*. IEEE. Available at: <https://ieeexplore.ieee.org/document/10778737> (Accessed: 2 December 2025)

Stryker, C. (n.d.) *What is a recurrent neural network?* IBM. Available at: <https://www.ibm.com/think/topics/recurrent-neural-networks> (Accessed: 2 December 2025)

Jayakannan, S. M. (2025) *Securing Voice-Based Financial Authentication in the Era of AI Voice Cloning*, Journal of Computer Science and Technology Studies, 7(4), pp. 515–520.

Stafford, G.A. (2025) *Fine-Tuning Wav2Vec2 for Real-Time Deepfake Audio Detection*, Data Science Collective. Available at: <https://medium.com/data-science-collective/fine-tuning-wav2vec2-for-real-time-deepfake-audio-detection-b72d7efebdd7>

ffmpeg.org. (n.d.). *Documentation*. [online] Available at: <https://ffmpeg.org/documentation.html>.

No-Code Start-Up. (2025). *How WhatsApp Business API Works: Official vs Unofficial*. [online] Available at: <https://nocodestartup.io/en/how-does-the-official-vs-unofficial-whatsapp-business-api-work-2/>

PyTorch (2024). *PyTorch documentation — PyTorch 2.7 documentation*. [online] Pytorch.org. Available at: <https://docs.pytorch.org/docs/stable/index.html>

Nccgroup.com. (2025). *The Rising Threat of Vishing Attacks and Deepfakes*. [online] Available at: <https://www.nccgroup.com/the-rising-threat-of-vishing-attacks-and-deepfakes/>

Huntress. (2026). *What is AI Phishing? Evolving Phishing Attacks in 2026 | Huntress*. [online] Available at: <https://www.huntress.com/generative-ai-guide/what-is-ai-phishing-evolving-phishing-attacks-in-2026>.

GeeksforGeeks (2024). *Long Short Term Memory (LSTM) Networks using PyTorch*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/deep-learning/long-short-term-memory-networks-using-pytorch/>

Esposito, P. (2020). *Building a LSTM by hand on PyTorch*. [online] Medium. Available at: <https://medium.com/data-science/building-a-lstm-by-hand-on-pytorch-59c02a4ec091>

Leupold, M.-D. (2024). *Transfer Learning & Finetuning for CNNs in PyTorch - Mark-Daniel Leupold - Medium*. [online] Medium. Available at: <https://medium.com/@marklpd/transfer-learning-finetuning-for-cnns-in-pytorch-5c4ade873d93>

kvpratama (2020). *Audio Classification with LSTM and TorchAudio*. [online] Kaggle.com. Available at: <https://www.kaggle.com/code/kvpratama/audio-classification-with-lstm-and-torchaudio>

Google Developers (2019). *Classification: ROC Curve and AUC | Machine Learning Crash Course*. [online] Google Developers. Available at: <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>

Baevski, A., Zhou, H., Mohamed, A. and Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *arXiv:2006.11477 [cs, eess]*. [online] Available at: <https://arxiv.org/abs/2006.11477>

Roberts, L. (2020). *Understanding the Mel Spectrogram*. [online] Medium. Available at: <https://medium.com/analytics-vidhya/understanding-the-mel-spectrogram-fca2afa2ce53>

Li, X., Chen, P.-Y. and Wei, W. (2024). *SONAR: A Synthetic AI-Audio Detection Framework and Benchmark*. [online] arXiv.org. Available at: <https://arxiv.org/abs/2410.04324>

GeeksforGeeks (2017). *Introduction to TensorFlow*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/python/introduction-to-tensorflow/>

Mintlify (2026). *ngrok documentation*. [online] Ngrok.com. Available at: <https://ngrok.com/docs/start>

Thor, W.-M. (2026). *Windowing Functions Explained*. [online] Apxml.com. Available at: <https://apxml.com/courses/introduction-to-speech-recognition/chapter-2-processing-audio-signals/windowing-functions-explained>

librosa.org. (n.d.). *librosa — librosa 0.8.0 documentation*. [online] Available at: <https://librosa.org/doc/latest/index.html>

Pytorch.org. (2023). *Torchaudio Documentation — Torchaudio 2.9.0 documentation*. [online] Available at: <https://docs.pytorch.org/audio/stable/index.html>

Nvidia.com. (2026). *CUDA Best Practices Guide — CUDA C++ Best Practices Guide 13.2 documentation*. [online] Available at: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#optimizing-cuda-applications>

Mahbub (2025). *What is Opus Audio Codec?* [online] Castr's Blog. Available at: <https://castr.com/blog/opus-audio-codec/>

Joshi, P. (2024). *'Transforming Sound: Audio Augmentation in Python'*. [online] Medium. Available at: <https://medium.com/@joshiprerak123/transforming-sound-audio-augmentation-in-python-89c1c08a836b>

docs.scipy.org. (n.d.). *scipy.signal.butter — SciPy v1.6.1 Reference Guide*. [online] Available at: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html>

Pykes, K. (2024). *AdamW Optimizer in PyTorch Tutorial*. [online] Datacamp.com. Available at: <https://www.datacamp.com/tutorial/adamw-optimizer-in-pytorch>

Sitanand (2025). *Modern Optimizers: AdamW, Lion, and What Actually Works at Scale*. [online] Medium. Available at: <https://medium.com/@spjosyula2005/modern-optimizers-adamw-lion-and-what-actually-works-at-scale-68ffc033713b>

developer.mozilla.org. (2023). *MediaDevices - Web APIs | MDN*. [online] Available at: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices>

Manyi (2024). *dtypes of tensors: bfloat16 vs float32 vs float16*. [online] Medium. Available at: <https://medium.com/@manyi.yim/bfloat16-vs-float32-vs-float16-back-to-the-basics-80d4aec49ca8>

Pytorch.org. (2016). *MelSpectrogram — Torchaudio 2.8.0 documentation*. [online] Available at:

<https://docs.pytorch.org/audio/main/generated/torchaudio.transforms.MelSpectrogram.html>

Ray, J. (2024). *The art of setting learning rate*. [online] Medium. Available at:

<https://medium.com/better-ml/the-art-of-setting-learning-rate-eff11ac0a737>

GeeksforGeeks (2024). *Softmax Activation Function in Neural Networks*. [online]

GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/deep-learning/the-role-of-softmax-in-neural-networks-detailed-explanation-and-applications/>

Pytorch.org. (2022). *Batched data loading with tensorclasses — tensordict 0.12 documentation*. [online] Available at:

https://docs.pytorch.org/tensordict/stable/tutorials/tensorclass_imagenet.html

GeeksforGeeks (2024). *What is Tensor and Tensor Shapes?* [online] GeeksforGeeks.

Available at: <https://www.geeksforgeeks.org/deep-learning/what-is-tensor-and-tensor-shapes/>

Hughes, C. (2022). *Demystifying PyTorch's WeightedRandomSampler by example*. [online]

Medium. Available at: <https://medium.com/data-science/demystifying-pytorchs-weightedrandomsampler-by-example-a68aceccb452>

Olamendy, J.C. (2023). *Back to Basics: Feature Extraction with CNN - Juan C Olamendy - Medium*. [online] Medium. Available at:

<https://medium.com/@juanc.olamendy/back-to-basics-feature-extraction-with-cnn-16b2d405011a>

OpenAI (2022). *Introducing Whisper*. [online] Openai.com. Available at:

<https://openai.com/index/whisper/>

ai.meta.com. (n.d.). *Wav2vec*. [online] Available at:

<https://ai.meta.com/research/impact/wav2vec/>

FastAPI (2023). *FastAPI*. [online] fastapi.tiangolo.com. Available at:

<https://fastapi.tiangolo.com/>

Amazon Web Services (2025). *Amazon Web Services (AWS) - Cloud Computing Services*.

[online] Amazon Web Services, Inc. Available at: <https://aws.amazon.com/>

Amazon Web Services (2024). *Amazon EC2*. [online] Amazon Web Services, Inc. Available

at: <https://aws.amazon.com/ec2/>

AWS (2025). *Build, Train, and Deploy Machine Learning Models | Amazon SageMaker*. [online] Amazon Web Services, Inc. Available at: <https://aws.amazon.com/sagemaker/>

Luna, J.C. (2025). *A Beginner's Guide to the Rectified Linear Unit (ReLU)*. [online] Datacamp.com. Available at: <https://www.datacamp.com/blog/rectified-linear-unit-relu>

Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W. and Plumbley, M.D. (2020). PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition. *arXiv:1912.10211 [cs, eess]*. [online] Available at: <https://arxiv.org/abs/1912.10211>